ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KHOA HỌC VÀ KỸ THUẬT THÔNG TIN



CHUYÊN ĐỀ TỐT NGHIỆP (IE400)

XÂY DỰNG WEB NHẬN DIỆN VÀ PHÂN LOẠI BỆNH LÝ CHO CÂY ỚT

Tên tiếng Anh: Developing a web application for chili plant disease detection and classification

Giảng viên hướng dẫn: ThS. Đỗ Minh Tiến

Sinh viên thực hiện: Tô Minh Hải – MSSV: 25410197

Ngành: Công nghệ Thông tin

TP. HỒ CHÍ MINH, 2026

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA CÔNG NGHỆ THÔNG TIN

Tô Minh Hải – MSSV: 25410197

CHUYÊN ĐỀ TỐT NGHIỆP (IE400)

XÂY DỰNG WEB NHẬN DIỆN VÀ PHÂN LOẠI BỆNH LÝ CHO CÂY ỚT

Tên tiếng Anh: Developing a web application for chili plant disease detection and classification

Ngành: Công nghệ Thông tin

GIẢNG VIÊN HƯỚNG DẪN
ThS. Đỗ Minh Tiến

TP. HỒ CHÍ MINH, 2026

THÔNG TIN HỘI ĐỒNG CHẤM CHUYÊN ĐỀ TỐT NGHIỆP (IE400)

Hội đồng chấm chuyên đề tốt nghiệp, thành lập theo Quyết định số ..... ngày ..... của Hiệu trưởng Trường Đại học Công nghệ Thông tin.

1. ..... – Chủ tịch.

2. ..... – Thư ký.

3. ..... – Ủy viên.

4. ..... – Ủy viên.

LỜI CAM ĐOAN

Em xin cam đoan chuyên đề tốt nghiệp "Xây dựng web nhận diện và phân loại bệnh lý cho cây ớt" là công trình nghiên cứu do chính em thực hiện dưới sự hướng dẫn của giảng viên hướng dẫn. Các số liệu, kết quả thực nghiệm trình bày trong chuyên đề là trung thực, được đo đạc trực tiếp từ mã nguồn và nhật ký huấn luyện của đề tài, không sao chép từ bất kỳ công trình nào khác. Các nội dung tham khảo từ tài liệu bên ngoài đều được trích dẫn rõ nguồn gốc trong phần Tài liệu tham khảo. Em xin chịu hoàn toàn trách nhiệm về tính trung thực của nội dung chuyên đề này.

TP. Hồ Chí Minh, ngày ..... tháng ..... năm 2026

Sinh viên thực hiện

Tô Minh Hải

LỜI CẢM ƠN

Chuyên đề này là kết quả của một chặng đường học tập tại Trường Đại học Công nghệ Thông tin, Đại học Quốc gia Thành phố Hồ Chí Minh. Em xin gửi lời cảm ơn chân thành đến Quý Thầy Cô của trường, đặc biệt là Khoa Công nghệ Thông tin, đã truyền đạt những kiến thức chuyên môn và tạo điều kiện để em hoàn thành chuyên đề này.

Em xin bày tỏ lòng biết ơn sâu sắc đến ThS. Đỗ Minh Tiến, giảng viên hướng dẫn, đã tận tình chỉ bảo, góp ý và định hướng trong suốt quá trình thực hiện đề tài, từ giai đoạn xây dựng bộ dữ liệu, thiết kế kiến trúc mô hình, cho đến khi hoàn thiện bài báo khoa học và chuyên đề này.

Do thời gian và kinh nghiệm còn hạn chế, chuyên đề khó tránh khỏi thiếu sót. Em rất mong nhận được sự góp ý của Quý Thầy Cô để hoàn thiện hơn nữa.

Em xin chân thành cảm ơn!

TP. Hồ Chí Minh, ngày ..... tháng ..... năm 2026

Sinh viên thực hiện

Tô Minh Hải

MỤC LỤC

MỞ ĐẦU 2
Lý do chọn đề tài 2
Mục tiêu nghiên cứu 3
Đối tượng và phạm vi nghiên cứu 4
Phương pháp nghiên cứu 4
Đóng góp của đề tài 5
Bố cục chuyên đề 6
Chương 1. CƠ SỞ LÝ THUYẾT 7
1.1. Bài toán phân loại ảnh bệnh cây trồng bằng học sâu 7
1.2. Mạng nơ-ron tích chập và kiến trúc MobileNetV2 7
1.3. Vision Transformer và cơ chế tự chú ý 8
1.4. Ba kỹ thuật tiết kiệm tham số vay mượn từ kiến trúc LLM 9
1.4.1. *Rotary Position Embedding (RoPE)* 9
1.4.2. *Grouped-Query Attention (GQA)* 10
1.4.3. *RMSNorm* 11
1.5. Cơ chế Triplet Attention và khối RES_Block 12
1.6. Các kỹ thuật AI giải thích được (XAI) 12
1.7. Phương pháp kiểm định thống kê 13
1.7.1. *Khoảng tin cậy Wilson* 13
1.7.2. *Kiểm định McNemar ghép cặp* 14
1.7.3. *Hiệu chỉnh Holm cho kiểm định nhiều giả thuyết* 14
Chương 2. CÁC NGHIÊN CỨU LIÊN QUAN 16

2.1. Tiêu chí lựa chọn nhóm baseline 16
2.2. CNN nhẹ cho phân loại bệnh cây trồng 17
2.3. Vision Transformer cho phân loại bệnh cây trồng 17
2.4. Kiến trúc lai CNN-Transformer 18
2.5. Các nghiên cứu so sánh diện rộng 18
2.6. Nghiên cứu phát hiện bệnh trên cây ớt 19
2.7. Thích nghi chuyển miền với ít mẫu 19
2.8. Khoảng trống nghiên cứu 20
Chương 3. PHƯƠNG PHÁP ĐỀ XUẤT 21
3.1. Bộ dữ liệu ChiliEdge-DS 21
3.2. Kiến trúc ChiliEdgeNet 22
3.2.1. *Điểm cắt xương sống MobileNetV2* 24
3.2.2. *Đường đi của đặc trưng qua mô hình* 24
3.2.3. *Khối Tiny Transformer và ba kỹ thuật tiết kiệm tham số* 24
3.2.4. *Loại bỏ Triplet Attention và RES_Block* 25
3.2.5. *Tổng số tham số mô hình* 26
3.3. Độ đo đánh giá 26
Chương 4. THỰC NGHIỆM VÀ KẾT QUẢ 28
4.1. Thiết lập thực nghiệm 28
4.2. So sánh với các mô hình SOTA 30
4.3. Phân tích ablation cấp khối 32
4.4. Stress test vai trò của khối Tiny Transformer trước suy giảm ảnh 34
4.5. Ablation cấp từng kỹ thuật 35
4.6. Đánh giá liên bộ dữ liệu sau thích nghi few-shot 36

4.7. Đo độ trễ tiền xử lý và suy luận trên ứng dụng camera laptop 39
4.8. Khảo sát mở rộng về nén mô hình 40
Chương 5. DIỄN GIẢI MÔ HÌNH 43
5.1. AI giải thích được (XAI) 43
5.2. Đối chiếu với một mô hình đa phương thức tổng quát 45
Chương 6. TRIỂN KHAI ỨNG DỤNG MINH HỌA 47
6.1. Kiến trúc tổng thể ứng dụng 47
6.1.1. *Tổ chức thành phần và luồng dữ liệu* 47
6.1.2. *Backend suy luận* 48
6.1.3. *Frontend* 49
6.1.4. *Sinh diễn giải kết quả bằng Gemini API* 50
6.1.5. *Đóng gói và triển khai* 51
6.2. Phân tích thiết kế hệ thống 51
6.2.1. *Biểu đồ use case* 52
6.2.2. *Biểu đồ tuần tự* 53
6.2.3. *Biểu đồ luồng dữ liệu* 54
6.3. Giao diện và luồng sử dụng 55
6.4. Chương trình demo thời gian thực trên camera laptop 59
6.4.1. *Cách hoạt động* 59
6.4.2. *Giao thức đo* 60
6.4.3. *Kết quả một phiên đo thực tế* 60
Chương 7. HẠN CHẾ VÀ HƯỚNG PHÁT TRIỂN 62
7.1. Hạn chế 62
7.2. Hướng phát triển 63

KẾT LUẬN 65
TÀI LIỆU THAM KHẢO 67

DANH MỤC HÌNH VẼ

Hình 3.1. Phân bố số ảnh theo lớp và theo tập train/val/test của ChiliEdge-DS. 22

Hình 3.2. Kiến trúc tổng thể ChiliEdgeNet và chi tiết một khối Tiny Transformer. 23

Hình 4.1. Ma trận nhầm lẫn của ChiliEdgeNet trên tập test (kịch bản augmented). 32

Hình 4.2. Độ chính xác của ChiliEdgeNet theo mức suy giảm ảnh, có và không có khối Tiny Transformer. 35

Hình 4.3. Ma trận nhầm lẫn của checkpoint gốc trên tập truy vấn sạch 1.463 ảnh. 38

Hình 4.4. Ma trận nhầm lẫn của checkpoint sau thích nghi few-shot trên cùng tập truy vấn sạch 1.463 ảnh. 39

Hình 4.5. Tổng hợp trực quan các phương án nén mô hình. 42

Hình 5.1. Occlusion Sensitivity cho năm lớp bệnh. 44

Hình 5.2. Gradient SHAP cho năm lớp. 44

Hình 5.3. Attention Rollout của khối Tiny Transformer. 45

Hình 6.1. Kiến trúc tổng thể của ứng dụng minh họa. 48

Hình 6.2. Biểu đồ use case của ứng dụng minh họa. 52

Hình 6.3. Biểu đồ tuần tự của một lượt chẩn đoán (/predict) và hỏi đáp (/chat). 54

Hình 6.4. Biểu đồ luồng dữ liệu (DFD) mức 1 của ứng dụng minh họa. 55

Hình 6.5. Trang chủ ứng dụng: thẻ thông tin mô hình, khung tải ảnh và kết quả dự đoán 56

Hình 6.6. Khối giải thích trực quan (XAI) trên giao diện ứng dụng. 56

Hình 6.7. Biểu đồ xếp hạng xác suất theo lớp và khối kiểm tra độ bền (Robustness). 57
Hình 6.8. Đoạn giải thích kết quả bằng ngôn ngữ tự nhiên do LLM sinh ra. 58
Hình 6.9. Khung hỏi đáp về bệnh cây, hỗ trợ bởi LLM. 58

DANH MỤC BẢNG

Bảng 3.1. Phân bố theo lớp của ChiliEdge-DS trên lần chia cố định. 21

Bảng 3.2. Siêu tham số khối Tiny Transformer. 26

Bảng 4.1. Cấu hình huấn luyện của ChiliEdgeNet và các biến thể ablation. 28

Bảng 4.2. Thay đổi khi thích nghi các kiến trúc baseline sang bài toán phân loại năm lớp. 29

Bảng 4.3. Độ chính xác hai kịch bản, macro-F1 và số tham số trên ChiliEdge-DS. 30

Bảng 4.4. Kết quả ablation kiến trúc trên ảnh sạch, seed 42. 32

Bảng 4.5. Kiểm định McNemar cho hai phép cắt trực tiếp trên ảnh sạch. 33

Bảng 4.6. Ba phép so sánh loại bỏ Triplet Attention/RES_Block khỏi cấu hình có cả hai. 33

Bảng 4.7. Stress test khối Tiny Transformer dưới các mức suy giảm tổng hợp. 34

Bảng 4.8. Ablation từng kỹ thuật trong khối Tiny Transformer trên ảnh sạch, seed 42. 35

Bảng 4.9. Phân chia tập hỗ trợ/truy vấn đã xác minh từ bốn lớp được ánh xạ. 37

Bảng 4.10. Kết quả trước và sau thích nghi few-shot trên tập truy vấn sạch 1.463 ảnh. 37

Bảng 4.11. Độ trễ tiền xử lý và suy luận của ChiliEdgeNet trên CPU laptop. 39

Bảng 4.12. Đường cong độ thưa–độ chính xác của cắt tỉa không cấu trúc. 40

Bảng 4.13. So sánh các phương án nén (đo trên cùng 373 ảnh test, kịch bản augmented, phiên đo riêng). 40

Bảng 5.1. So sánh ChiliEdgeNet với Gemini 3.1 Flash Lite trên 373 ảnh test. 45

Bảng 6.1. Tổng hợp phiên demo camera laptop. 60

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Tiếng Anh	Giải thích (tiếng Việt)
API	Application Programming Interface	Giao diện lập trình ứng dụng
CBAM	Convolutional Block Attention Module	Mô-đun chú ý khối tích chập
CI	Confidence Interval	Khoảng tin cậy
CNN	Convolutional Neural Network	Mạng nơ-ron tích chập
CPU	Central Processing Unit	Bộ xử lý trung tâm
DFD	Data Flow Diagram	Biểu đồ luồng dữ liệu
DS	Dataset	Bộ dữ liệu
ECA	Efficient Channel Attention	Chú ý kênh hiệu quả
F1	F1-score	Điểm F1 (trung bình điều hòa Precision/Recall)
FP32/FP16	32/16-bit Floating Point	Số thực dấu phẩy động 32/16-bit
GELU	Gaussian Error Linear Unit	Hàm kích hoạt tuyến tính sai số Gauss
GPU	Graphics Processing Unit	Bộ xử lý đồ họa
GQA	Grouped-Query Attention	Chú ý truy vấn theo nhóm
INT8	8-bit Integer	Số nguyên 8-bit

IoT	Internet of Things	Internet vạn vật
LLM	Large Language Model	Mô hình ngôn ngữ lớn
LSTM	Long Short-Term Memory	Mạng bộ nhớ dài-ngắn hạn
MHA	Multi-Head Attention	Chú ý đa đầu
MLP	Multi-Layer Perceptron	Mạng perceptron đa lớp
PE	Positional Embedding	Nhúng vị trí
pp	percentage point	Điểm phần trăm
REST	Representational State Transfer	Kiểu kiến trúc giao tiếp API qua HTTP
RMSNorm	Root Mean Square Normalization	Chuẩn hóa theo căn bậc hai trung bình bình phương
ROI	Region of Interest	Vùng quan tâm
RoPE	Rotary Position Embedding	Mã hóa vị trí quay
SE	Squeeze-and-Excitation	Khối nén và kích hoạt kênh
SHAP	SHapley Additive exPlanations	Giải thích cộng tính dựa trên giá trị Shapley
SOTA	State Of The Art	Hiện đại nhất / dẫn đầu
SVM	Support Vector Machine	Máy vectơ hỗ trợ
ViT	Vision Transformer	Bộ biến đổi thị giác
XAI	Explainable AI	AI giải thích được

TÓM TẮT CHUYÊN ĐỀ

Chẩn đoán bệnh cây trồng cho các thiết bị hạn chế tài nguyên đòi hỏi một mô hình vừa chính xác vừa nhẹ. Chuyên đề này đề xuất ChiliEdgeNet, một kiến trúc lai CNN-Transformer được xây dựng bằng cách cắt ngắn MobileNetV2 và tích hợp một khối Tiny Transformer. Chuyên đề cũng giới thiệu ChiliEdge-DS, bộ dữ liệu công bố công khai gồm 2.461 ảnh lá và quả ớt thuộc năm lớp bệnh/trạng thái. ChiliEdgeNet đạt 98,93% độ chính xác với 671.173 tham số, ít hơn khoảng 35% so với kiến trúc nền tảng. Mô hình có dung lượng 2,7 MB và độ trễ suy luận 20,24 ms/ảnh trên một CPU hạng máy chủ. Trong phép kiểm thử suy giảm độ sáng tổng hợp ở hệ số 0,2, mô hình đầy đủ vượt biến thể không có khối Transformer 3,75 điểm phần trăm. Trên một tập truy vấn chéo bộ dữ liệu gồm 1.463 ảnh sạch thuộc bốn lớp chung với bộ dữ liệu Mendeley Data, độ chính xác tăng từ 30,96% ở checkpoint gốc lên 89,75% sau khi thích nghi bằng 42 ảnh hỗ trợ; cả hai checkpoint đều dùng seed 42. Các kết quả này cho thấy ChiliEdgeNet là một ứng viên khả thi cho suy luận ngoại tuyến trên phần cứng hạn chế tài nguyên, nhưng việc đánh giá trực tiếp trên phần cứng biên thật và xác nhận thêm về khả năng chuyển miền trên nhiều bộ dữ liệu độc lập hơn vẫn còn cần thiết.

Chuyên đề trình bày cơ sở lý thuyết cho từng kỹ thuật được sử dụng: CNN, MobileNetV2, Vision Transformer, RoPE, GQA, RMSNorm, các kỹ thuật XAI và kiểm định thống kê có hiệu chỉnh Holm. Mỗi thành phần của mô hình đều được kiểm tra riêng. Các biến thể được huấn luyện lại từ đầu sau khi bỏ từng thành phần, rồi so với mô hình đầy đủ bằng kiểm định McNemar trên từng ảnh. Chuyên đề cũng đo độ bền của mô hình khi ảnh bị nhòe hoặc thiếu sáng, đánh giá khả năng thích nghi trên một bộ dữ liệu độc lập, và dùng ba kỹ thuật XAI thuộc ba nhóm cơ chế để xem mô hình dựa vào vùng nào của ảnh.

MỞ ĐẦU

Lý do chọn đề tài

Cây ớt là một trong những loại cây trồng phổ biến ở nhiều vùng nông nghiệp Việt Nam, mang lại giá trị kinh tế đáng kể cho người nông dân. Nhưng cây ớt dễ mắc nhiều loại bệnh (thán thư, xoăn lá, đốm lá, vàng lá), và nếu phát hiện trễ, năng suất lẫn chất lượng thu hoạch đều giảm mạnh. Vấn đề là phần lớn nông dân không có sẵn chuyên gia để hỏi ngay lúc phát hiện triệu chứng, còn các giải pháp AI đám mây thì cần kết nối Internet ổn định, thứ không phải khu vực canh tác nào cũng có. Một mô hình chẩn đoán ngay tại điểm chụp ảnh, vì vậy, cần đáp ứng cùng lúc ba điều: chi phí tính toán thấp, chạy được ngoại tuyến, và vẫn giữ độ chính xác đủ cao cho thiết bị hạn chế tài nguyên.

Học sâu, đặc biệt là mạng nơ-ron tích chập (CNN), đã cho kết quả tốt trong bài toán phân loại bệnh cây trồng từ ảnh [27], [35]. Các mạng tích chập nhẹ có ít tham số và mang thiên kiến quy nạp cục bộ (local inductive bias), phù hợp với đặc trưng thị giác. Hạn chế của chúng là mỗi lớp chỉ tổng hợp thông tin trong một vùng nhỏ. Mạng sâu có thể mở rộng trường nhận thức, nhưng vẫn không mô hình hóa trực tiếp quan hệ giữa mọi cặp vị trí trong ảnh như cơ chế tự chú ý (self-attention). Điều này ảnh hưởng tới các bệnh như xoăn lá hay vàng lá, vì vùng bệnh trải khắp bề mặt lá. Vision Transformer xử lý tốt quan hệ toàn cục, song các cấu hình tiêu chuẩn tốn nhiều tính toán hơn CNN nhẹ. Kiến trúc lai CNN-Transformer là một cách dung hòa hai hướng này.

Các công trình khảo sát ở Chương 2 báo cáo kết quả trên những bộ dữ liệu và quy trình khác nhau. Một số bài phân tích đóng góp của từng thành phần kiến trúc, một số đo độ bền trước ảnh suy giảm, một số xét khoảng cách miền. Điều còn thiếu là một quy trình gộp các phân tích đó trên cùng một lần chia dữ liệu và cùng một công thức huấn luyện. Ngoài ra, độ chính xác cao trên ảnh có nền được kiểm soát chưa đảm bảo mô hình chạy tốt trên ảnh nền tự nhiên. Đề tài hướng tới khoảng trống này theo hai hướng. Thứ nhất, xây dựng một mô hình lai CNN-Transformer đủ nhẹ

để chạy trên thiết bị biên. Thứ hai, kiểm tra từng lựa chọn thiết kế bằng ablation huấn luyện lại từ đầu, kiểm định thống kê có hiệu chỉnh cho nhiều phép so sánh, và đánh giá chuyển miền trên một bộ dữ liệu độc lập.

Kế thừa trực tiếp từ kiến trúc RTR_Lite_MobileNet [7], vốn dựa trên MobileNetV2 [33], đề tài này đề xuất ChiliEdgeNet, một mô hình lai CNN-Transformer dưới 700 nghìn tham số.

Mục tiêu nghiên cứu

Đề tài đặt ra các mục tiêu cụ thể sau:

1. Xây dựng và công bố bộ dữ liệu ảnh bệnh cây ớt tự thu thập (ChiliEdge-DS), phục vụ huấn luyện và đánh giá mô hình.

2. Thiết kế một kiến trúc lai CNN-Transformer nhẹ (dưới 700 nghìn tham số) cho bài toán phân loại 5 lớp bệnh/trạng thái cây ớt, có khả năng chạy ngoại tuyến trên CPU với độ trễ thấp.

3. Tích hợp và đánh giá định lượng ba kỹ thuật tiết kiệm tham số vay mượn từ kiến trúc mô hình ngôn ngữ lớn (LLM), gồm 2D-RoPE, GQA và RMSNorm, vào khối Transformer thị giác.

4. Đánh giá mô hình một cách toàn diện: so sánh với các mô hình SOTA đã phục dựng dưới cùng quy trình, đo độ bền trước điều kiện chụp suy giảm (nhòe, thiếu sáng), và thực hiện ablation huấn luyện lại từ đầu ở cả cấp khối lẫn cấp từng kỹ thuật, kèm kiểm định thống kê có hiệu chỉnh Holm cho việc kiểm định nhiều giả thuyết trên cùng dữ liệu.

5. Đánh giá khả năng chuyển miền của mô hình bằng thích nghi few-shot trên một bộ dữ liệu bệnh cây ớt độc lập, thu thập ở một quốc gia khác.

6. Diễn giải quyết định của mô hình bằng các kỹ thuật AI giải thích được (XAI) thuộc nhiều nhóm cơ chế khác nhau, và đối chiếu độc lập với một mô hình đa phương thức tổng quát.

Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu là ảnh lá và quả ớt thuộc năm lớp: anthracnose (thán thư), healthy (khỏe), leaf_curl (xoăn lá), leaf_spot (đốm lá), yellow_leaf (vàng lá).

Phạm vi nghiên cứu:

- Về dữ liệu: bộ dữ liệu ChiliEdge-DS tự thu thập thực địa tại khu vực Củ Chi, Thành phố Hồ Chí Minh, ảnh được chụp trên phông nền trắng có kiểm soát, khác với ảnh điện thoại chụp trực tiếp ngoài đồng ruộng với nền lá/đất phức tạp. Khoảng cách miền này được đánh giá một phần thông qua benchmark suy giảm (mục 4.4) và một phần thông qua đánh giá thích nghi trên bộ dữ liệu độc lập từ Bangladesh (mục 4.6), nhưng chưa được đánh giá đầy đủ trên ảnh nền tự nhiên, một hạn chế được nêu rõ ở Chương 7. Nhãn được gán thủ công dựa trên biểu hiện hình thái quan sát bằng mắt (phenotype-based), không phải kết quả xác định tác nhân gây bệnh bằng xét nghiệm phòng thí nghiệm.
- Về mô hình: tập trung vào bài toán phân loại ảnh, không mở rộng sang phát hiện đối tượng hay phân vùng.
- Về đánh giá: các kết quả ablation và so sánh với kiến trúc nền tảng đều dựa trên một lần chạy (seed 42) kèm kiểm định thống kê ghép cặp có hiệu chỉnh Holm, không phải trung bình nhiều seed; giới hạn này được trình bày minh bạch ở Chương 7.

Phương pháp nghiên cứu

Đề tài kết hợp nghiên cứu lý thuyết và thực nghiệm định lượng:

- **Nghiên cứu lý thuyết:** khảo sát cơ sở lý thuyết về CNN, MobileNetV2, Vision Transformer, các kỹ thuật tiết kiệm tham số trong LLM (2D-RoPE, GQA, RMSNorm), và các kỹ thuật XAI thuộc nhiều nhóm cơ chế khác nhau.
- **Nghiên cứu thực nghiệm:** cài đặt và huấn luyện mô hình đề xuất cùng 17 baseline phục dựng từ 15 nghiên cứu liên quan, theo cùng một quy trình huấn luyện chung (cùng dữ liệu, cùng lần chia, cùng seed, cùng bộ độ đo) để đảm

bảo so sánh công bằng nhất có thể trong điều kiện mỗi baseline vẫn giữ thiết kế tái tạo riêng (kích thước ảnh vào, batch size, epoch, optimizer, cường độ tăng cường dữ liệu) theo đúng mô tả gốc của từng nghiên cứu.

- **Phương pháp kiểm định thống kê:** dùng kiểm định McNemar ghép cặp từng ảnh để đánh giá ý nghĩa thống kê của chênh lệch giữa hai mô hình trên cùng tập test, khoảng tin cậy Wilson 95% để định lượng độ bất định của từng giá trị độ chính xác, và hiệu chỉnh Holm (Holm-Bonferroni) khi thực hiện nhiều phép kiểm định trên cùng một họ so sánh.
- **Phương pháp ablation:** huấn luyện lại từ đầu (không phải knock-out tại thời điểm suy luận) cho từng biến thể kiến trúc, nhằm đo đúng *đóng góp* của từng thành phần thay vì mức độ *phụ thuộc* của mạng đã huấn luyện vào thành phần đó.
- **Phương pháp đánh giá chuyển miền:** thích nghi có giám sát bằng một tập hỗ trợ nhỏ (few-shot supervised domain adaptation) trên một bộ dữ liệu bệnh cây ớt độc lập, kiểm soát rò rỉ dữ liệu bằng băm SHA-256 giữa tập hỗ trợ và tập truy vấn.

Đóng góp của đề tài

- **ChiliEdgeNet**: mô hình lai CNN-Transformer chạy hoàn toàn ngoại tuyến trên CPU ở 20,24 ms/ảnh (2,7 MB), đạt 98,93% độ chính xác với 671.173 tham số, ít hơn khoảng 35% so với kiến trúc nền tảng RTR_Lite_MobileNet [7] (1.032.568 tham số) cùng quy trình seed 42.
- Xây dựng và công bố ChiliEdge-DS, bộ dữ liệu bệnh ớt tự thu thập thực địa tại Củ Chi, Thành phố Hồ Chí Minh (5 lớp, 2.461 ảnh, chia sẵn cố định train/val/test).
- Một cuộc đánh giá độ bền có kiểm soát và đánh giá liên bộ dữ liệu: ablation huấn luyện lại từ đầu ở cấp khối và cấp từng kỹ thuật dưới điều kiện suy giảm tổng hợp, cùng đánh giá khả năng chuyển miền trên một tập truy vấn sạch gồm 1.463 ảnh thuộc bốn lớp từ bộ dữ liệu Mendeley Data: độ chính xác tăng

từ 30,96% ở checkpoint gốc lên 89,75% sau thích nghi bằng chỉ 42 ảnh hỗ trợ, với tập hỗ trợ và tập truy vấn không giao nhau (xác nhận bằng băm SHA-256).

- Diễn giải mô hình bằng ba kỹ thuật XAI thuộc ba nhóm cơ chế khác nhau (Occlusion Sensitivity, Attention Rollout, Gradient SHAP), cùng một đối chứng zero-shot với một mô hình đa phương thức tổng quát trên toàn bộ 373 ảnh test.
- Tích hợp mô hình vào một ứng dụng camera laptop và đo độ trễ tiền xử lý cộng suy luận trên phần cứng CPU tiêu dùng, như một minh chứng khả thi cho triển khai thực tế.

Bố cục chuyên đề

Phần còn lại của chuyên đề được tổ chức thành bảy chương, phần kết luận và tài liệu tham khảo. Chương 1 trình bày cơ sở lý thuyết về CNN, MobileNetV2, Vision Transformer, ba kỹ thuật tiết kiệm tham số vay mượn từ LLM, cơ chế Triplet Attention, các kỹ thuật XAI, và phương pháp kiểm định thống kê được dùng xuyên suốt chuyên đề. Chương 2 khảo sát các nghiên cứu liên quan và xác định khoảng trống nghiên cứu. Chương 3 trình bày phương pháp đề xuất: bộ dữ liệu ChiliEdge-DS và kiến trúc ChiliEdgeNet. Chương 4 trình bày thực nghiệm và kết quả: so sánh SOTA, ablation cấp khối và cấp kỹ thuật, benchmark suy giảm, đánh giá chuyển miền few-shot, đo độ trễ trên ứng dụng camera, và một khảo sát mở rộng về khả năng nén mô hình. Chương 5 diễn giải mô hình bằng XAI và đối chiếu với một mô hình đa phương thức tổng quát. Chương 6 giới thiệu ứng dụng web minh họa triển khai mô hình, kèm các biểu đồ use case, tuần tự và luồng dữ liệu của ứng dụng. Chương 7 trình bày các hạn chế và hướng phát triển. Cuối cùng là phần Kết luận và Tài liệu tham khảo.

Chương 1. CƠ SỞ LÝ THUYẾT

Phần lý thuyết dưới đây đi theo thứ tự các thành phần của mô hình, từ xương sống MobileNetV2 tới khối Transformer, sau đó là các kỹ thuật XAI và các phép kiểm định thống kê dùng ở Chương 4.

1.1. Bài toán phân loại ảnh bệnh cây trồng bằng học sâu

Phân loại ảnh bệnh cây trồng là bài toán học có giám sát: cho một ảnh đầu vào x, mô hình f_θ cần dự đoán nhãn lớp $\hat{y} = f_\theta(x) \in \{1, \ldots, K\}$ trong đó K là số lớp bệnh/trạng thái. Trước khi học sâu trở nên phổ biến, các phương pháp truyền thống dựa trên trích xuất đặc trưng thủ công (màu sắc, kết cấu, hình dạng vết bệnh) kết hợp bộ phân loại như SVM. Học sâu, đặc biệt là mạng nơ-ron tích chập (Convolutional Neural Network, CNN), thay thế bước trích xuất đặc trưng thủ công bằng các lớp tích chập học được trực tiếp từ dữ liệu, cho phép mô hình tự động học các đặc trưng phân cấp, từ cạnh, kết cấu ở lớp nông đến hình dạng vết bệnh phức tạp ở lớp sâu.

Hàm mất mát phổ biến cho bài toán phân loại nhiều lớp là cross-entropy:

$$L = -\sum_{k=1}^{K} y_k \log \hat{y}_k$$

trong đó y_k là nhãn thật dạng one-hot và $\hat{y}_k$ là xác suất dự đoán cho lớp k sau lớp softmax cuối cùng. Toàn bộ mô hình trong chuyên đề này (bao gồm ChiliEdgeNet và 17 baseline phục dựng) đều được huấn luyện end-to-end với hàm mất mát này, có làm mượt nhãn (label smoothing, xem Bảng 4.1).

1.2. Mạng nơ-ron tích chập và kiến trúc MobileNetV2

Lớp tích chập trượt một bộ lọc (kernel) qua ảnh đầu vào, tính tích chập cục bộ tại từng vị trí để tạo bản đồ đặc trưng (feature map). So với lớp kết nối đầy đủ, tích chập chia sẻ trọng số trên toàn bộ không gian ảnh, giảm đáng kể số tham số và khai thác tính bất biến dịch chuyển (translation invariance) của đặc trưng thị giác. Tuy nhiên, mỗi phép tích chập chỉ nhìn thấy một vùng cục bộ (trường nhận thức, receptive

field) tại một thời điểm; để nắm được quan hệ giữa các vùng ảnh cách xa nhau, mạng cần xếp chồng nhiều lớp tích chập hoặc dùng lớp gộp (pooling) để mở rộng dần trường nhận thức. Một mạng đủ sâu vẫn có thể đạt trường nhận thức lớn, nhưng ở mỗi lớp, phép tích chập vẫn chỉ tổng hợp thông tin cục bộ chứ không trực tiếp mô hình hóa quan hệ theo cặp giữa mọi vị trí như tự chú ý.

MobileNetV2 [33] là kiến trúc CNN nhẹ được thiết kế cho thiết bị di động, dựa trên hai ý tưởng chính:

- **Depthwise separable convolution**: một phép tích chập chuẩn được tách thành hai bước là depthwise convolution (mỗi kênh đầu vào được tích chập riêng biệt bằng một bộ lọc không gian) và pointwise convolution (tích chập 1×1 để trộn thông tin giữa các kênh). Cách tách này giảm đáng kể số phép tính và số tham số so với tích chập chuẩn cùng kích thước, đặc biệt khi số kênh đầu ra lớn.
- **Inverted residual với linear bottleneck**: khác với khối residual truyền thống (nén số kênh ở giữa), MobileNetV2 mở rộng số kênh ở lớp giữa (theo hệ số giãn nở, expansion ratio) rồi nén lại ở đầu ra bằng một phép chiếu tuyến tính không có hàm kích hoạt phi tuyến (linear bottleneck), nhằm tránh mất thông tin khi biểu diễn đi qua không gian nhiều chiều bị nén bởi ReLU. Mỗi khối InvertedResidual có một kết nối tắt (skip connection) nối thẳng đầu vào tới đầu ra khi kích thước phù hợp; mục 4.3 (Chương 4) sẽ đo bằng thực nghiệm xem việc vô hiệu hóa các kết nối tắt này có làm giảm độ chính xác hay không.

ChiliEdgeNet (Chương 3) sử dụng MobileNetV2 tiền huấn luyện trên ImageNet làm xương sống trích xuất đặc trưng, nhưng chỉ giữ lại các khối tích chập đầu thay vì toàn bộ mạng, với lý do trình bày chi tiết ở mục 3.2.1.

1.3. Vision Transformer và cơ chế tự chú ý

Kiến trúc Transformer, ban đầu được đề xuất cho xử lý ngôn ngữ tự nhiên [36], được Vision Transformer (ViT) [6] đưa vào thị giác máy tính bằng cách chia ảnh thành các ô lưới (patch) và coi mỗi ô như một "token", tương tự một từ trong câu

văn bản. Mỗi token được chiếu tuyến tính thành một vector nhúng (embedding), cộng thêm thông tin vị trí (positional encoding) vì cơ chế tự chú ý vốn không có khái niệm thứ tự không gian.

Với mỗi token, cơ chế tự chú ý tính một tổ hợp có trọng số của mọi token khác trong chuỗi. Cụ thể, mỗi token được chiếu thành ba vector: truy vấn (query), khóa (key) và giá trị (value). Độ tương đồng giữa vector truy vấn của token đang xét và vector khóa của mọi token khác (chuẩn hóa qua hàm softmax) quyết định trọng số dùng để tổ hợp các vector giá trị tương ứng. Multi-head attention chạy song song nhiều "đầu" chú ý với các phép chiếu truy vấn/khóa/giá trị khác nhau, cho phép mô hình học nhiều kiểu quan hệ khác nhau cùng lúc, rồi ghép kết quả lại.

Một khối Transformer chuẩn gồm một lớp tự chú ý và một mạng truyền thẳng nhiều lớp (MLP), mỗi lớp đều có kết nối tắt (residual connection) và chuẩn hóa (normalization) trước hoặc sau. Ưu điểm chính so với CNN là trường nhận thức toàn cục ngay từ lớp đầu tiên (mọi token có thể chú ý trực tiếp đến mọi token khác), phù hợp để nắm bắt các triệu chứng bệnh lan tỏa trên toàn bộ lá. Nhược điểm là chi phí của tự chú ý tăng theo bình phương số token. Transformer cũng thiếu thiên kiến quy nạp cục bộ như tích chập, nên thường cần nhiều dữ liệu hơn [6], hoặc phải đặt sau một xương sống CNN đã học sẵn đặc trưng cục bộ. ChiliEdgeNet chọn cách thứ hai.

1.4. Ba kỹ thuật tiết kiệm tham số vay mượn từ kiến trúc LLM

Các mô hình ngôn ngữ lớn (Large Language Model, LLM) hiện đại đã phát triển nhiều kỹ thuật để giảm số tham số và bộ nhớ mà không đánh đổi nhiều về chất lượng, do quy mô hàng tỉ tham số khiến mọi khoản tiết kiệm đều có giá trị lớn. Ba kỹ thuật sau đây, vốn thiết kế cho Transformer văn bản, được tích hợp vào khối Tiny Transformer thị giác của ChiliEdgeNet.

1.4.1. *Rotary Position Embedding (RoPE)*

Positional embedding học được (learned positional embedding) là phương án phổ biến nhất trong ViT gốc: một bảng tra cứu kích thước (số token × số chiều nhúng)

được học cùng mô hình. Nhược điểm là tốn tham số và bị giới hạn cứng bởi số token đã cố định lúc huấn luyện.

RoPE [34] mã hóa vị trí bằng cách xoay từng cặp thành phần liên tiếp của vector truy vấn/khóa theo một góc tỉ lệ thuận với vị trí của token trong chuỗi, thay vì cộng thêm một vector học được như positional embedding truyền thống: về bản chất, mỗi cặp thành phần được nhân với một ma trận quay hai chiều mà góc quay phụ thuộc vị trí token.

Tính chất quan trọng của phép quay này là mức độ tương đồng giữa truy vấn ở một vị trí và khóa ở vị trí khác chỉ phụ thuộc vào vị trí tương đối giữa hai token, không phụ thuộc vị trí tuyệt đối. Nhờ vậy, vị trí tương đối được mã hóa mà không cần thêm tham số học nào. Với dữ liệu ảnh (lưới hai chiều), biến thể RoPE trục cho Vision Transformer [16] áp dụng phép quay độc lập trên từng trục của lưới: một nửa số chiều của mỗi đầu chú ý được xoay theo tọa độ hàng, nửa còn lại theo tọa độ cột, sao cho vị trí tương đối được mã hóa riêng theo cả hai chiều không gian.

Ưu điểm cốt lõi so với positional embedding học được: RoPE không tốn một tham số học nào, và không bị giới hạn bởi số token cố định lúc huấn luyện. Chương 3 áp dụng kỹ thuật này cho lưới đặc trưng 14×14 (196 token) của ChiliEdgeNet, và Chương 4 đo định lượng khoản tiết kiệm tham số so với phương án học được.

1.4.2. *Grouped-Query Attention (GQA)*

Trong multi-head attention chuẩn (MHA), mỗi đầu chú ý có bộ trọng số truy vấn, khóa, giá trị riêng, với H đầu, cần H bộ trọng số khóa/giá trị. GQA [3] chia H đầu truy vấn thành G nhóm ($G < H$), mỗi nhóm chia sẻ chung một cặp đầu khóa/giá trị, trong khi mỗi đầu truy vấn trong nhóm vẫn giữ trọng số truy vấn riêng. Trường hợp đặc biệt $G = 1$ là Multi-Query Attention (mọi đầu truy vấn chia sẻ một cặp khóa/giá trị duy nhất); trường hợp $G = H$ chính là MHA chuẩn.

GQA ban đầu được thiết kế để giảm dung lượng bộ nhớ đệm khóa/giá trị (KV-cache) khi LLM sinh văn bản tuần tự; bộ nhớ này tỉ lệ thuận với số đầu khóa/giá trị. Trong bối cảnh một bộ phân loại ảnh không có sinh tuần tự, lợi ích về bộ nhớ đó

không còn ý nghĩa, nhưng khoản tiết kiệm tham số từ việc giảm số bộ trọng số khóa/giá trị vẫn có thật; lợi ích này được định lượng như một khoản giảm tham số chiếu (projection parameters), không được suy rộng thành một tuyên bố về tốc độ suy luận. ChiliEdgeNet dùng bốn đầu truy vấn chia sẻ hai đầu khóa/giá trị (*H = 4, G = 2*); Chương 4 đo xem khoản tiết kiệm này có phải trả giá bằng độ chính xác hay không.

1.4.3. *RMSNorm*

LayerNorm, phép chuẩn hóa phổ biến trong Transformer, chuẩn hóa mỗi vector đặc trưng bằng cách trừ trung bình và chia cho độ lệch chuẩn (gọi là tái tâm hóa, re-centering, và tái tỉ lệ, re-scaling), sau đó áp dụng một phép biến đổi affine học được gồm một hệ số nhân và một hệ số cộng (bias).

RMSNorm [43] giả thuyết rằng phần tái tâm hóa không phải là yếu tố quan trọng nhất của LayerNorm, mà chính phần tái tỉ lệ mới đóng vai trò ổn định huấn luyện. RMSNorm bỏ bước trừ trung bình và bỏ luôn tham số dịch (bias), chỉ chuẩn hóa mỗi vector theo căn bậc hai trung bình bình phương (root mean square) của chính nó rồi nhân với hệ số học được:

$$\mathrm{RMSNorm}(x) = \frac{x}{\mathrm{RMS}(x)} \odot \gamma, \quad \mathrm{RMS}(x) = \sqrt{\frac{1}{n}\sum_{i=1}^{n} x_i^2 + \varepsilon}$$

Vì không còn tham số dịch, RMSNorm tiết kiệm đúng bằng số chiều của vector đặc trưng so với LayerNorm, cộng thêm lợi ích tính toán nhỏ (bỏ bước tính trung bình). Đây là một khác biệt cấu trúc chính xác về mặt số tham số; tác động của RMSNorm lên độ chính xác vẫn cần được xác nhận bằng ablation thay vì suy luận trực tiếp từ số tham số tiết kiệm được. Mục 4.5 kiểm tra tác động của khoản tiết kiệm nhỏ này (384 tham số trong ChiliEdgeNet) bằng ablation.

1.5. Cơ chế Triplet Attention và khối RES_Block

Triplet Attention [28] là một mô-đun chú ý nhẹ, gần như không cần tham số học, tính toán tương tác chéo giữa ba cặp chiều của tensor đặc trưng (kênh với chiều cao, kênh với chiều rộng, chiều cao với chiều rộng) thông qua ba nhánh xoay tensor và nén (Z-pool) song song, rồi tổng hợp lại. Ý tưởng cốt lõi là các mô-đun chú ý phổ biến trước đó (ví dụ Squeeze-and-Excitation [18]) thường chỉ tính tương tác giữa các kênh mà bỏ qua tương tác không gian, trong khi Triplet Attention tính cả ba chiều tương tác cùng lúc với chi phí thấp.

Kiến trúc nền tảng RTR_Lite_MobileNet [7] mà đề tài dựa vào đặt một cổng Triplet Attention và một khối RES_Block, tức khối residual kết hợp Efficient Channel Attention (ECA) [38] với Squeeze-and-Excitation (SE) [18], ngay sau lớp tích chập đầu của MobileNetV2. ChiliEdgeNet bỏ cả hai khối này (mục 3.2.4); bằng chứng ablation cho quyết định đó nằm ở mục 4.3.

1.6. Các kỹ thuật AI giải thích được (XAI)

Độ chính xác cao chưa đủ để tin cậy một mô hình phân loại ảnh: mô hình có thể đạt độ chính xác cao nhờ bám vào manh mối giả (ví dụ đặc điểm phông nền) thay vì đặc trưng bệnh thật sự. Rủi ro này càng đáng lưu ý vì phần lớn ảnh trong ChiliEdge-DS được chụp trên nền trắng. Các kỹ thuật AI giải thích được (Explainable AI, XAI) giúp trực quan hóa vùng ảnh mà mô hình dựa vào để ra quyết định, chia thành các nhóm cơ chế khác nhau:

- **Nhóm nhiễu loạn (perturbation-based)**: Occlusion Sensitivity [42] che từng vùng ảnh 32×32 pixel bằng một mảng màu trung tính rồi đo mức thay đổi đầu ra, hoàn toàn không nhìn vào bên trong mô hình (không gradient, không trọng số, không kích hoạt), nên là bằng chứng độc lập với mọi giả định về kiến trúc, có thể áp dụng cho bất kỳ mô hình hộp đen nào.
- **Nhóm dựa trên gradient/kích hoạt**: Gradient SHAP, cụ thể là phương pháp Expected Gradients [8], ước lượng đóng góp từng pixel bằng giá trị kỳ vọng của gradient khi lấy baseline từ một phân phối, thay vì một baseline cố định

như Integrated Gradients. Đây là phương pháp gán thuộc tính thỏa các tiên đề (axiomatic attribution) như tính đầy đủ, được Erion và cộng sự đề xuất để xây dựng attribution prior giúp cải thiện hiệu năng mô hình, và cho kết quả xấp xỉ giá trị Shapley.

- **Nhóm đọc nội tại Transformer**: Attention Rollout [1] đọc trực tiếp ma trận trọng số chú ý bên trong các khối Transformer đã huấn luyện, nhân dồn (rollout) qua các lớp để truy vết dòng chú ý từ token đầu ra ngược về lưới token đầu vào, tạo bản đồ cho biết token nào ảnh hưởng nhiều nhất đến quyết định cuối cùng. Vì ChiliEdgeNet dùng average pooling theo token thay vì token CLS, bản đồ Rollout được tạo bằng cách lấy trung bình dòng chú ý trên toàn bộ 196 token thay vì truy vết từ một token đầu ra duy nhất.

Vì mỗi nhóm dựa trên một tập giả định khác nhau về cách "nhìn" vào mô hình, sự đồng thuận giữa các kỹ thuật thuộc nhiều nhóm cơ chế khác nhau là bằng chứng mạnh hơn nhiều so với các kỹ thuật cùng nhóm (vốn dễ cùng sai theo một kiểu hệ thống). Chương 5 áp dụng nguyên tắc này, dùng ba kỹ thuật thuộc ba nhóm cơ chế khác nhau làm bằng chứng định tính: Occlusion Sensitivity, Gradient SHAP và Attention Rollout.

1.7. Phương pháp kiểm định thống kê

Với một tập test kích thước hữu hạn (373 ảnh trong chuyên đề này), chênh lệch nhỏ giữa hai mô hình (ví dụ 1–2 ảnh) có thể chỉ là nhiễu ngẫu nhiên của cách chia dữ liệu, không phản ánh một khác biệt kiến trúc thật sự. Ba công cụ thống kê được dùng xuyên suốt Chương 4 để phân biệt tín hiệu thật khỏi nhiễu ngẫu nhiên.

1.7.1. *Khoảng tin cậy Wilson*

Với một tỉ lệ quan sát được $\hat{p}$ (ví dụ độ chính xác) trên n mẫu, khoảng tin cậy Wilson [39] cho ước lượng khoảng tin cậy chính xác hơn khoảng Wald (chuẩn) thông thường khi $\hat{p}$ gần 0 hoặc gần 1, đúng trường hợp độ chính xác quanh 98–99% trong chuyên đề này:

$$\hat{p}_{\text{Wilson}} = \frac{\hat{p} + \frac{z^2}{2n} \pm z\sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}}$$

với z = 1,96 ứng với độ tin cậy 95%. Mọi giá trị độ chính xác trong Chương 4 đều kèm khoảng tin cậy Wilson 95% ở n = 373, giúp người đọc đánh giá đúng mức độ chắc chắn của từng con số thay vì chỉ nhìn một giá trị điểm.

1.7.2. *Kiểm định McNemar ghép cặp*

Khi so sánh hai mô hình được đánh giá trên cùng một tập ảnh test, kiểm định McNemar [25] là công cụ phù hợp hơn nhiều so với so sánh trực tiếp hai giá trị độ chính xác, vì nó khai thác thông tin ghép cặp (mô hình nào đúng/sai trên từng ảnh cụ thể) thay vì chỉ so sánh hai tỉ lệ tổng hợp độc lập. Với bảng liền kề 2×2 giữa hai mô hình trên cùng tập ảnh:

	Mô hình B đúng	**Mô hình B sai**
Mô hình A đúng	*a*	*b*
Mô hình A sai	*c*	*d*

chỉ hai ô lệch nhau *b* (A đúng, B sai) và *c* (A sai, B đúng) mang thông tin phân biệt hai mô hình; *a* và *d* (hai mô hình đồng ý) không đóng góp. Với *b*+*c* nhỏ, chuyên đề dùng kiểm định nhị thức chính xác (exact binomial test) trên *b* trong *b*+*c* phép thử với xác suất kỳ vọng 0,5, để tránh sai số của phép xấp xỉ chi bình phương khi cỡ mẫu vừa/nhỏ. Mỗi phép so sánh được báo cáo dưới dạng cặp *b/c* và giá trị *p*: *b* là số ảnh mô hình thứ nhất đúng còn mô hình thứ hai sai, *c* là ngược lại.

1.7.3. *Hiệu chỉnh Holm cho kiểm định nhiều giả thuyết*

Khi cùng một tập dữ liệu được dùng để thực hiện nhiều phép kiểm định McNemar liên tiếp (ví dụ so sánh mô hình đề xuất với nhiều biến thể ablation, hoặc trên nhiều điều kiện suy giảm khác nhau), xác suất ít nhất một phép kiểm định cho kết quả "có ý nghĩa" hoàn toàn do ngẫu nhiên sẽ tăng lên theo số phép kiểm định thực hiện (vấn đề so sánh nhiều lần, multiple comparisons). Hiệu chỉnh Bonferroni đơn

giản (chia ngưỡng α cho số phép kiểm định) là một cách kiểm soát vấn đề này nhưng khá bảo thủ.

Chuyên đề dùng hiệu chỉnh Holm [17] (Holm-Bonferroni, còn gọi là phương pháp step-down): sắp xếp m giá trị p thu được theo thứ tự tăng dần $p_{(1)} \leq p_{(2)} \leq ... \leq p_{(m)}$, so sánh $p_{(1)}$ với ngưỡng α/m, $p_{(2)}$ với ngưỡng $\alpha/(m-1)$, và cứ thế tăng dần cho tới khi gặp giá trị không còn vượt qua ngưỡng tương ứng; mọi giá trị từ đó trở đi coi như không có ý nghĩa. So với Bonferroni đơn giản (chia đều α/m cho mọi phép kiểm định), Holm mạnh hơn (ít bảo thủ hơn) trong khi vẫn kiểm soát chặt tỉ lệ sai lầm loại I trên toàn họ so sánh (family-wise error rate). Giá trị p đã hiệu chỉnh theo phương pháp này được ký hiệu p_{Holm} trong Chương 4; một phép so sánh chỉ được coi là có ý nghĩa thống kê nếu $p_{Holm} < 0,05$.

Chương 2. CÁC NGHIÊN CỨU LIÊN QUAN

Chương này khảo sát các nghiên cứu gần đây về phân loại bệnh cây trồng dựa trên ảnh. Mục 2.1 nêu tiêu chí chọn nhóm baseline dùng để so sánh ở Chương 4; các mục 2.2 đến 2.5 trình bày bốn hướng nghiên cứu của nhóm baseline; các mục 2.6 và 2.7 bổ sung những nghiên cứu trên cây ớt và về thích nghi chuyển miền; mục 2.8 tổng hợp khoảng trống nghiên cứu mà đề tài hướng tới.

2.1. Tiêu chí lựa chọn nhóm baseline

Nhóm baseline so sánh ở Bảng 4.3 (Chương 4) gồm 17 baseline đến từ 15 nghiên cứu, được chọn theo bốn tiêu chí: (i) công trình bình duyệt công bố 2023-2026 về phân loại bệnh cây trồng dựa trên ảnh; (ii) có trọng tâm kiến trúc rõ ràng về khả năng triển khai (CNN nhẹ / ViT / lai) hoặc là nghiên cứu so sánh diện rộng CNN-ViT; (iii) mô tả kiến trúc đủ chi tiết (cấu hình lớp, số chiều nhúng, số đầu chú ý) để phục dựng hoặc thích nghi cho bài toán mục tiêu mà không cần liên hệ tác giả; (iv) phủ đủ bốn hướng nghiên cứu trình bày ở các mục 2.2 đến 2.5 thay vì chỉ chọn họ mạnh nhất. Các mục 2.6 và 2.7 bổ sung thêm những nghiên cứu liên quan trực tiếp tới cây ớt và tới thích nghi chuyển miền; các công trình này được dùng làm bối cảnh cho đề tài, không nằm trong nhóm baseline. Các nghiên cứu bị loại khi hướng tới phát hiện/phân vùng thay vì phân loại, hoặc khi không thể dựng lại một đầu ra phân loại tương ứng từ mô tả trong bài báo gốc. Với mô hình đa nhiệm, kiến trúc gốc được giữ lại nếu có một nhánh phân loại tường minh có thể thích nghi được, chẳng hạn PDLC-ViT [15] giữ nguyên phần co-scale, co-attention, cross-attention và bộ mã hóa Transformer, chỉ loại bỏ nhánh phân vùng/định vị và chuyển sang phân loại năm lớp thuần túy.

Nhóm baseline vì vậy đại diện cho dòng nghiên cứu hướng triển khai trong ba năm gần đây, không phải một khảo sát toàn diện của toàn bộ lĩnh vực; mỗi mô hình đều tự phục dựng theo một khung đánh giá chung (Bảng 4.1, Chương 4): cùng bộ dữ liệu ChiliEdge-DS, cùng lần chia cố định, cùng seed 42, cùng bộ độ đo; nhưng các

siêu tham số phụ thuộc kiến trúc (kích thước ảnh vào, batch size, số epoch, optimizer, cường độ tăng cường dữ liệu) vẫn theo đúng thiết kế tái tạo riêng của từng mô hình.

2.2. CNN nhẹ cho phân loại bệnh cây trồng

Nhóm nghiên cứu này giữ số tham số thấp bằng các khối chú ý hiệu quả hoặc cấu trúc khối gọn nhẹ. RTR_Lite_MobileNet [7], kiến trúc nền tảng mà đề tài này dựa vào và cải tiến, gắn thêm Triplet Attention [28] và một khối RES_Block kết hợp ECA với SE vào xương sống MobileNetV2 [33]. ULEN [37] là mạng siêu nhẹ dùng tích chập depth-wise có kết nối tắt và spatial pyramid pooling, bài gốc báo cáo khoảng 100 nghìn tham số (bản phục dựng năm lớp ở Bảng 4.3 còn 28.069 tham số), phù hợp các thiết bị cực kỳ hạn chế tài nguyên. MRW-CNN [20] và Lite-MDC [5] hướng tới triển khai thời gian thực ngoài đồng ruộng, đi kèm bộ dữ liệu tự thu thập riêng của từng nghiên cứu. CustomMNetV3L [31] tùy biến MobileNetV3-Large cho bài toán phân loại bệnh cây trồng. Cả nhóm đều thuần tích chập, nên chịu giới hạn về trường nhận thức đã nêu ở mục 1.2.

2.3. Vision Transformer cho phân loại bệnh cây trồng

Nhóm này dùng tự chú ý để mô hình hóa quan hệ giữa các token (mục 1.3), đổi lại chi phí tính toán thường cao hơn CNN nhẹ. ViT [4] áp dụng kiến trúc ViT gốc [6] cho bài toán bệnh cây trồng, chia ảnh thành các patch cố định và xử lý như chuỗi token. TrIncNet [9] thay khối MLP trong bộ mã hóa ViT bằng một khối inception (gọi là khối Trans-Inception), mỗi khối có kết nối tắt bao quanh. PMVT [23] dựa trên MobileViT, thay tích chập bằng kernel 7×7 và thêm mô-đun chú ý CBAM, hướng tới chạy trên thiết bị di động. MaizeFormerX [32] là một ViT nhẹ dùng patch embedding đa tỉ lệ (4, 8 và 16 pixel) kết hợp mô-đun chú ý xuyên tỉ lệ (cross-scale attention) để nắm cả chi tiết vết bệnh lẫn cấu trúc lan rộng. PDLC-ViT [15] là một mô hình đa nhiệm, vừa định vị (localization) vừa phân loại bệnh, làm tăng độ phức tạp và số tham số so với các mô hình chỉ phân loại thuần túy; trong Bảng 4.3, biến thể chỉ giữ nhánh phân loại được dùng để so sánh công bằng hơn với các mô hình còn lại. Ngoài nhóm

baseline, Hassan và cộng sự [14] đưa khối inception đa kernel vào ViT để tăng độ đa dạng của đặc trưng trích xuất cho bài toán nhận dạng bệnh trên lá.

2.4. Kiến trúc lai CNN-Transformer

Nhóm kiến trúc lai gần nhất với ChiliEdgeNet kết hợp một xương sống CNN trích xuất đặc trưng cục bộ với một khối Transformer nắm bắt quan hệ toàn cục. GrapeLeafNet [22] hợp nhất Inception-ResNet với Shuffle-Transformer qua hai nhánh song song. LDI-NET [40] kết hợp bộ token hóa đặc trưng (Feature Tokenizer) và bộ mã hóa token (Token Encoder) cho bài toán đa nhãn, dự đoán đồng thời loại cây, loại bệnh và mức độ nghiêm trọng; khi phục dựng, bộ giải mã đa nhãn được thay bằng average pooling toàn cục và một lớp phân loại tuyến tính năm lớp để phù hợp với bài toán mục tiêu. Cả hai mô hình này đều có số tham số trong khoảng 5–9 triệu (theo bản phục dựng ở Bảng 4.3), nặng hơn ChiliEdgeNet (671.173 tham số) nhiều lần.

Một số thiết kế lai công bố gần đây cũng đi theo hướng kết hợp đặc trưng cục bộ và toàn cục nhưng không nằm trong nhóm baseline phục dựng ở Bảng 4.3: ConvTransNet-S [19] nhắm tới nhận dạng bệnh trong môi trường đồng ruộng có nền phức tạp; Mehdipour và cộng sự [26] đề xuất một mô hình lai CNN-ViT nhẹ cho bệnh lá ngô; Ahmmed và cộng sự [2] kết hợp CNN với bộ mã hóa Transformer và tích hợp sẵn các kỹ thuật AI giải thích được cho bệnh lá cây dược liệu. Các công trình này củng cố xu hướng lai CNN-Transformer, song mỗi công trình được đánh giá trên bộ dữ liệu và giao thức riêng nên không so sánh trực tiếp được với nhau.

2.5. Các nghiên cứu so sánh diện rộng

Một số nghiên cứu không đề xuất kiến trúc mới mà tập trung đối chuẩn (benchmark) nhiều họ mô hình sẵn có. Mishra và cộng sự [27] so sánh một CNN tự thiết kế với các họ CNN cổ điển AlexNet, ResNet và EfficientNet trên bài toán phân loại bệnh khoai tây và xoài; do bài gốc không nêu biến thể cụ thể, khi phục dựng cho ChiliEdge-DS đề tài chọn AlexNet, ResNet-50 và EfficientNetB0 làm đại diện và

huấn luyện lại từ đầu theo cùng khung đánh giá. Sujatha và cộng sự [35] ghép đặc trưng sâu trích từ nhiều backbone (VGG19, InceptionV3) với nhiều bộ phân loại truyền thống (SVM, kNN, Random Forest, AdaBoost, Decision Tree); khi phục dựng, mười tổ hợp giữa backbone và bộ phân loại được đánh giá trên cùng lần chia dữ liệu, và cấu hình InceptionV3 kết hợp SVM (đạt độ chính xác test cao nhất trong số đó) được chọn làm đại diện ở Bảng 4.3; vì lựa chọn này dựa trên kết quả test, đây là một phân tích khám phá và không dùng để xếp hạng. Maurya và cộng sự [24] xây dựng một mô hình meta-ensemble nhẹ kết hợp MLP-Mixer, một nhánh CNN-LSTM và một bộ phân loại meta SVM, hướng tới triển khai trên môi trường IoT; khi phục dựng, đầu ra được đổi thành năm lớp và cấu hình patch-embedding được cài đặt theo mô tả có sẵn do bài báo gốc không cung cấp đầy đủ chi tiết.

2.6. Nghiên cứu phát hiện bệnh trên cây ớt

Riêng với cây ớt, bài toán chẩn đoán bệnh bằng ảnh cũng đã được quan tâm ở nhiều vùng trồng khác nhau. Kanaparthi và Ilango [21] xây dựng một khung học sâu phát hiện và phân loại kịp thời bệnh và sâu hại trên lá ớt. Odounfa và cộng sự [30] nghiên cứu ứng dụng học sâu cho nông nghiệp chính xác nhằm phát hiện bệnh ớt tại Benin. Hai công trình này cho thấy tính cấp thiết của bài toán, nhưng không hướng tới bối cảnh chạy ngoại tuyến trên thiết bị hạn chế tài nguyên mà đề tài nhắm đến.

2.7. Thích nghi chuyển miền với ít mẫu

Độ chính xác cao trên một bộ dữ liệu không bảo đảm mô hình giữ được hiệu năng khi điều kiện thu nhận ảnh thay đổi. Zhao và cộng sự [44] đề xuất nhận dạng bệnh cây few-shot chuyển miền với mô hình nhẹ bằng cách hiệu chỉnh thống kê theo miền đích; Yang và cộng sự [41] nghiên cứu học few-shot chuyển miền từ ảnh chụp trong phòng thí nghiệm sang ảnh ngoài đồng ruộng cho bài toán nhận dạng bệnh cây trồng. Mục 4.6 xuất phát từ cùng động cơ này, nhưng giới hạn ở phép ánh xạ bốn lớp đặc thù cho cây ớt giữa ChiliEdge-DS và một bộ dữ liệu độc lập thu thập tại Bangladesh [29].

2.8. Khoảng trống nghiên cứu

Từ khảo sát trên, ba khoảng trống nổi bật định hướng cho đề tài này. Về kiến trúc, các mô hình lai hiện có đạt độ chính xác tốt nhưng đều nặng hơn đáng kể so với nhóm CNN nhẹ thuần túy, chưa khai thác các kỹ thuật tiết kiệm tham số đã được kiểm chứng trong lĩnh vực LLM. Phương pháp đánh giá cũng là một vấn đề: phần lớn công trình xếp hạng mô hình chỉ bằng độ chính xác trên ảnh sạch của một lần chia dữ liệu, không có ablation kiểm soát phương sai, không kiểm định thống kê ghép cặp có hiệu chỉnh cho việc kiểm định nhiều lần, và không đánh giá độ bền trước điều kiện chụp thực tế (nhòe, thiếu sáng) hay khả năng chuyển sang một bộ dữ liệu độc lập khác. Cuối cùng là tính minh bạch của thiết kế. Các công trình thường báo cáo mô hình hoàn chỉnh mà không đo xem từng thành phần có xứng với số tham số nó chiếm hay không. Vì vậy khó biết thành phần nào thật sự cần, thành phần nào chỉ làm mô hình phức tạp thêm.

ChiliEdgeNet, trình bày ở Chương 3, giải quyết khoảng trống thứ nhất bằng cách tích hợp ba kỹ thuật tiết kiệm tham số từ LLM vào một khối Transformer thị giác cỡ nhỏ đặt sau một xương sống MobileNetV2 được cắt ngắn có chủ đích. Chương 4 và Chương 5 giải quyết khoảng trống thứ hai và thứ ba bằng benchmark điều kiện suy giảm, đánh giá chuyển miền few-shot trên bộ dữ liệu độc lập, kiểm định McNemar ghép cặp từng ảnh có hiệu chỉnh Holm, và ablation huấn luyện lại từ đầu ở cả cấp khối lẫn cấp từng kỹ thuật.

Chương 3. PHƯƠNG PHÁP ĐỀ XUẤT

Chương này trình bày phương pháp đề xuất, gồm bộ dữ liệu ChiliEdge-DS do tác giả tự thu thập (mục 3.1), kiến trúc ChiliEdgeNet cùng lý do cho từng lựa chọn thiết kế (mục 3.2), và các độ đo dùng để đánh giá mô hình (mục 3.3).

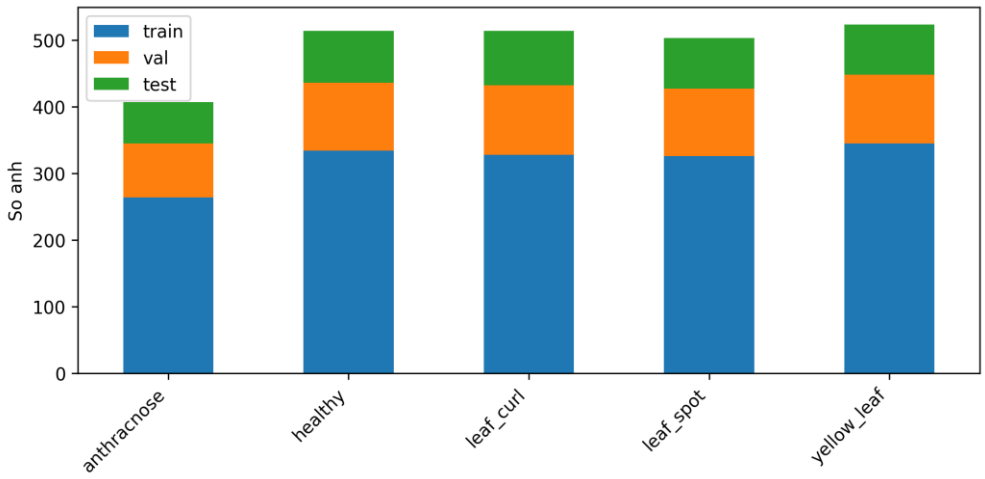
3.1. Bộ dữ liệu ChiliEdge-DS

ChiliEdge-DS gồm 2.461 ảnh lá và quả ớt thuộc năm lớp: anthracnose (thán thư), healthy (khỏe), leaf_curl (xoăn lá), leaf_spot (đốm lá), yellow_leaf (vàng lá), tự thu thập thực địa tại khu vực Củ Chi, Thành phố Hồ Chí Minh. Dữ liệu được chia một lần theo tỉ lệ xấp xỉ 65:20:15 thành các tập train/val/test gồm 1.597 / 491 / 373 ảnh (Bảng 3.1), và giữ nguyên xuyên suốt toàn bộ chuyên đề để kết quả phản ánh đúng khả năng tổng quát hóa trên dữ liệu chưa thấy.

Ảnh được chụp trên phông nền trắng bằng camera điện thoại thông minh (không ghi lại model thiết bị cụ thể) để làm nổi bật vùng bệnh, ở độ phân giải gốc khoảng 12.000×9.000 pixel, được lưu ở kích thước 300×300 pixel rồi resize về 224×224 khi đưa vào mô hình.

Bảng 3.1. Phân bố theo lớp của ChiliEdge-DS trên lần chia cố định.

Lớp	**Train**	**Val**	**Test**	**Tổng**
anthracnose	264	81	62	407
healthy	334	102	78	514
leaf_curl	328	104	82	514
leaf_spot	326	101	76	503
yellow_leaf	345	103	75	523
Tổng	**1.597**	**491**	**373**	**2.461**



Hình 3.1. Phân bố số ảnh theo lớp và theo tập train/val/test của ChiliEdge-DS.

Nhãn được gán thủ công dựa trên biểu hiện quan sát bằng mắt: anthracnose thể hiện các vết hoại tử sẫm màu trên bề mặt quả; healthy không có triệu chứng quan sát được rõ ràng; leaf_curl thể hiện lá bị cong hoặc xoắn; leaf_spot thể hiện các đốm tổn thương rời rạc trên bề mặt lá; yellow_leaf thể hiện lá úa vàng trên diện rộng. Đây là các nhãn dựa trên kiểu hình quan sát trực quan, không phải kết quả xác định tác nhân gây bệnh đã được xác nhận bằng xét nghiệm phòng thí nghiệm.

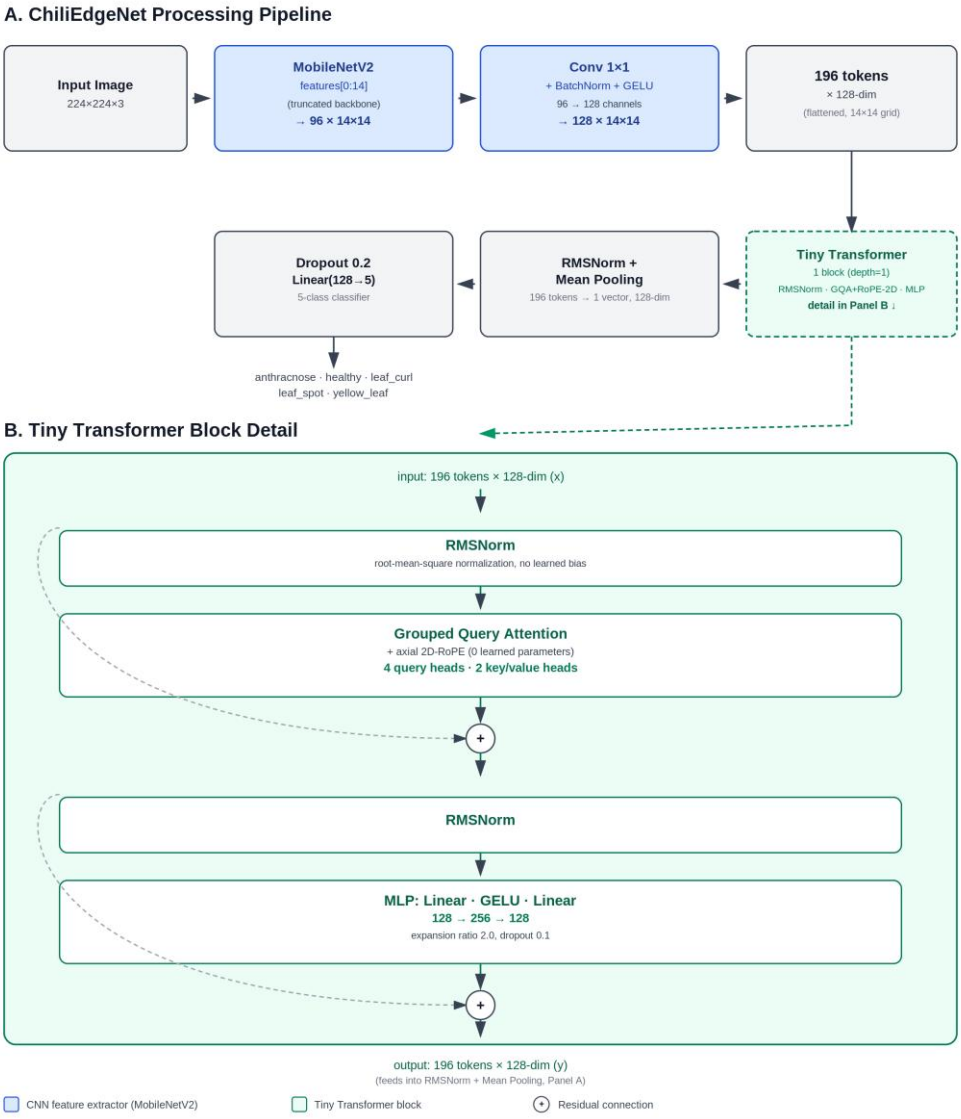
Tỉ lệ giữa lớp lớn nhất và nhỏ nhất là 523/407 = 1,29 (bộ dữ liệu gần cân bằng), nên chuyên đề không áp dụng class weighting khi huấn luyện, và macro-F1 (mục 3.3) được báo cáo song song với độ chính xác để mỗi lớp đóng góp ngang nhau vào chỉ số tổng hợp, không bị lớp đa số chi phối. Nền trắng là một điều kiện thu thập có kiểm soát, khác với ảnh điện thoại chụp trực tiếp ngoài đồng ruộng với nền lá/đất phức tạp; mục 4.4 đánh giá một phần khoảng cách này thông qua các phép suy giảm áp ở thời điểm kiểm thử, mục 4.6 đánh giá thêm khả năng chuyển miền trên một bộ dữ liệu độc lập, và mục 7.1 nêu rõ phần chưa được đánh giá đầy đủ.

ChiliEdge-DS được công bố công khai trên Google Drive [13].

3.2. Kiến trúc ChiliEdgeNet

Triệu chứng xoăn lá và vàng lá trải khắp bề mặt lá, nên mô hình cần nắm được quan hệ giữa các vùng cách xa nhau. CNN làm tốt phần đặc trưng cục bộ (mục 1.2),

còn tự chú ý xử lý tốt quan hệ tầm xa (mục 1.3). ChiliEdgeNet vì thế kết hợp một xương sống MobileNetV2 cắt ngắn với một khối Tiny Transformer (Hình 3.2). So với kiến trúc nền tảng [7], ChiliEdgeNet cũng loại bỏ Triplet Attention [28] và khối RES_Block kết hợp ECA với SE; mục 4.3 trình bày bằng chứng ablation cho lựa chọn này.



Hình 3.2. Kiến trúc tổng thể ChiliEdgeNet và chi tiết một khối Tiny Transformer.

Khối A mô tả luồng xử lý tổng thể: ảnh 224×224×3 qua xương sống MobileNetV2 đã cắt ngắn (features[0:14]), một tích chập 1×1, 196 token, khối Tiny Transformer, mean pooling và bộ phân loại, kèm kích thước tensor tại từng bước;

khối B nêu chi tiết một khối Tiny Transformer, gồm RMSNorm, Grouped-Query Attention với 2D-RoPE trục (4 đầu truy vấn, 2 đầu khóa/giá trị) có kết nối tắt, và một MLP dùng GELU (tỉ lệ mở rộng 2,0) có kết nối tắt.

3.2.1. *Điểm cắt xương sống MobileNetV2*

ChiliEdgeNet dùng MobileNetV2 tiền huấn luyện trên ImageNet, giữ lại các lớp từ features[0] đến features[13]. Với ảnh vào 224×224, xương sống đã cắt cho ra bản đồ đặc trưng kích thước 96×14×14, tương ứng 196 vị trí không gian đưa vào khối Transformer như 196 token ảnh.

Điểm cắt được đặt trước features[14] vì tầng này giảm bản đồ đặc trưng xuống 7×7, tức số token giảm từ 196 còn 49. Ở độ phân giải 7×7, các phép tích chập phía trước đã tích hợp thông tin trên gần như toàn bộ ảnh vào mỗi ô lưới, khiến khối Transformer phía sau gần như không còn quan hệ không gian thật sự nào để học thêm. Điểm cắt cut = 14 được chọn thông qua một phép quét sơ bộ trên các điểm cắt còn giữ được lưới 14×14, và còn mang lại lợi ích trực tiếp cho khả năng diễn giải: bản đồ Attention Rollout ở Chương 5 có 196 vị trí không gian, gấp bốn lần một lưới 7×7, đủ mịn để khoanh vùng triệu chứng thay vì chỉ chỉ ra một góc phần tư của ảnh.

3.2.2. *Đường đi của đặc trưng qua mô hình*

Luồng xử lý của ChiliEdgeNet gồm các bước sau: ảnh đầu vào đi qua các khối tích chập đầu của MobileNetV2 (features[0:14]); một lớp tích chập 1×1 ánh xạ 96 kênh xuống 128 kênh; BatchNorm, GELU và dropout 0,1; làm phẳng bản đồ đặc trưng thành 196 token ảnh, mỗi token 128 chiều; khối Tiny Transformer; RMSNorm và lấy trung bình theo chiều token (mean pooling) để thu về một vector 128 chiều duy nhất; cuối cùng, dropout 0,2 rồi một lớp tuyến tính dự đoán năm lớp.

3.2.3. *Khối Tiny Transformer và ba kỹ thuật tiết kiệm tham số*

Bất chấp tên gọi có yếu tố "Former" gợi liên tưởng ngôn ngữ, ChiliEdgeNet hoàn toàn là một mô hình thị giác và không chứa thành phần ngôn ngữ nào: không bộ mã hóa văn bản, không tokenizer, không trọng số ngôn ngữ tiền huấn luyện. Mô

hình được huấn luyện end-to-end chỉ trên ảnh với hàm mất mát cross-entropy (mục 1.1), như một bộ phân loại thông thường.

Khối Tiny Transformer dùng cấu trúc pre-norm: RMSNorm, tiếp theo là GQA kết hợp với 2D-RoPE trục và một kết nối tắt, tiếp theo là RMSNorm, một MLP dùng GELU, và một kết nối tắt thứ hai. Mô hình dùng số chiều nhúng 128, bốn đầu truy vấn, hai đầu khóa/giá trị, tỉ lệ mở rộng MLP bằng 2, và dropout 0,1.

Ba kỹ thuật tiết kiệm tham số (lý thuyết tại mục 1.4) được áp dụng cụ thể như sau:

- 2D-RoPE trục (mục 1.4.1) xoay vector truy vấn/khóa theo góc phụ thuộc tọa độ hàng/cột trên lưới 14×14, thay thế một bảng positional embedding học được, tiết kiệm 25.088 tham số (196×128).
- GQA (mục 1.4.2) cho bốn đầu truy vấn chia sẻ hai đầu khóa/giá trị, tiết kiệm 16.384 tham số chú ý so với multi-head attention chuẩn bốn đầu khóa/giá trị riêng biệt.
- RMSNorm (mục 1.4.3) bỏ phần trừ trung bình và tham số dịch của LayerNorm, tiết kiệm 384 tham số.

Ba lựa chọn này cộng lại tiết kiệm 41.856 tham số, khoảng 6,2% số tham số của ChiliEdgeNet. Đây là các khác biệt cấu trúc chính xác về mặt đếm tham số; tác động của từng kỹ thuật lên độ chính xác được đo riêng bằng ablation huấn luyện lại từ đầu ở mục 4.5 (Chương 4), không được suy luận trực tiếp từ số tham số tiết kiệm được.

3.2.4. *Loại bỏ Triplet Attention và RES_Block*

Song song với việc cắt gọn tham số ở khối Transformer, ChiliEdgeNet không giữ lại hai khối kế thừa từ kiến trúc nền tảng [7]: Triplet Attention [28] và khối RES_Block kết hợp ECA với SE (mục 1.5), cả hai vốn đặt ngay sau lớp tích chập đầu của MobileNetV2. Ở giai đoạn đầu, quyết định này chỉ dựa trên một khảo sát sơ bộ. Mục 4.3 kiểm tra lại bằng thiết kế thêm lại kiểu 2×2 (cả hai khối, chỉ Triplet, chỉ RES_Block, không khối nào) kèm kiểm định McNemar có hiệu chỉnh Holm.

3.2.5. *Tổng số tham số mô hình*

Với cấu hình trên, ChiliEdgeNet có tổng cộng 671.173 tham số, khoảng 35,0% ít hơn so với kiến trúc nền tảng RTR_Lite_MobileNet [7] (1.032.568 tham số ở bản phục dựng năm lớp; bài gốc báo cáo 1.050.547 tham số), tương ứng dung lượng lưu trữ khoảng 2,7 MB ở định dạng FP32. Bảng 3.2 tổng hợp siêu tham số của khối Tiny Transformer.

Bảng 3.2. Siêu tham số khối Tiny Transformer.

Siêu tham số	**Giá trị**
Số chiều nhúng (embedding dim)	128
Số lớp Transformer	1
Số token đầu vào	196 (lưới 14×14)
Số đầu truy vấn (GQA)	4
Số đầu khóa/giá trị (GQA)	2
Tỉ lệ mở rộng MLP	2
Dropout (khối Transformer / đầu ra)	0,1 / 0,2
Mã hóa vị trí	2D-RoPE trục (không tham số)
Chuẩn hóa	RMSNorm
Tổng tham số toàn mô hình	671.173

3.3. Độ đo đánh giá

Chuyên đề dùng các độ đo sau, thống nhất xuyên suốt Chương 4:

- **Accuracy (độ chính xác top-1)**: tỉ lệ ảnh được dự đoán đúng trên tổng số N ảnh.
- Precision, Recall và F1-score cho từng lớp, tính từ số dương tính thật (TP), dương tính giả (FP) và âm tính giả (FN):

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \quad F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

- **Macro-F1**: trung bình không trọng số của F1 từng lớp trên năm lớp, để mỗi lớp đóng góp ngang nhau bất kể kích thước.
- Tổng số tham số (Total Parameters), đo bằng công cụ torchinfo.summary().

- Khoảng tin cậy Wilson 95% (công thức tại mục 1.7.1) đi kèm mọi giá trị độ chính xác trên tập test 373 ảnh.
- Kiểm định McNemar ghép cặp từng ảnh có hiệu chỉnh Holm (công thức tại mục 1.7.2–1.7.3) cho mọi so sánh giữa hai mô hình trên cùng tập ảnh.

Chương 4. THỰC NGHIỆM VÀ KẾT QUẢ

Mọi thí nghiệm trong chương dùng cùng một lần chia dữ liệu và seed 42. Các so sánh chính đi kèm khoảng tin cậy Wilson và kiểm định McNemar có hiệu chỉnh Holm (mục 1.7).

4.1. Thiết lập thực nghiệm

Thí nghiệm chạy trên Kaggle với GPU NVIDIA Tesla T4; mã phục dựng dùng PyTorch hoặc TensorFlow tùy theo kiến trúc gốc của từng nghiên cứu. Mỗi baseline được đánh giá theo hai kịch bản: kịch bản baseline không áp dụng tăng cường dữ liệu, và kịch bản augmented áp dụng các kỹ thuật tăng cường ở Bảng 4.1. Tập val và tập test không bao giờ được tăng cường ở cả hai kịch bản; với các mô hình huấn luyện theo epoch, checkpoint có tiêu chí validation tốt nhất luôn được nạp lại trước khi đánh giá trên tập test.

Khung đánh giá chung. Cả 17 baseline đều dùng chung bộ dữ liệu ChiliEdge-DS, cùng lần chia train/val/test cố định, cùng seed 42, và cùng bộ độ đo độ chính xác/macro-F1/số tham số. Cả hai kịch bản đều được đánh giá trên cùng tập test không tăng cường. Các thiết lập phụ thuộc kiến trúc (kích thước ảnh vào, batch size, số epoch, optimizer, cường độ tăng cường dữ liệu) theo đúng thiết kế phục dựng riêng của từng mô hình. Phép so sánh vì vậy được kiểm soát về dữ liệu, lần chia và độ đo, nhưng không dùng một công thức huấn luyện đồng nhất tuyệt đối cho mọi kiến trúc.

Bảng 4.1. Cấu hình huấn luyện của ChiliEdgeNet và các biến thể ablation.

Mục	**Giá trị**
Ảnh vào	224×224
Transform baseline	Resize, ToTensor, Normalize
Transform augmented	Xoay 30°, ColorJitter, lật ngang, biến đổi affine
Epoch	30
Batch size	32
Hàm mất mát	Cross-entropy, label smoothing 0,05
Lịch học	Warmup tuyến tính 10% → cosine decay
Tốc độ học (backbone / khối mới)	1e-4 / 1e-3
Seed	42 (mọi biến thể)

Hai trường hợp lựa chọn baseline chịu ảnh hưởng bởi kết quả test: RTR_Lite_MobileNet [7] được chọn làm baseline chính, và cấu hình InceptionV3 + SVM được chọn làm đại diện cho nghiên cứu của Sujatha và cộng sự [35]. Điều này được trình bày minh bạch và diễn giải như phân tích khám phá ở mục 4.2, không phải một đánh giá test độc lập. Bảng 4.2 tóm tắt các thay đổi đáng kể khi thích nghi bốn kiến trúc baseline có cấu trúc đầu ra khác biệt sang bài toán năm lớp.

Bảng 4.2. Thay đổi khi thích nghi các kiến trúc baseline sang bài toán phân loại năm lớp.

Baseline	**Thành phần giữ nguyên**	**Thành phần được điều chỉnh**
PDLC-ViT (chỉ phân loại; [15])	Co-scale, co-attention, cross-attention, bộ mã hóa Transformer, đầu phân loại	Loại bỏ nhánh phân vùng/định vị; chuyển từ đa nhiệm sang phân loại năm lớp thuần túy, chỉ dùng loss phân loại; các siêu tham số ViT chưa công bố đầy đủ được thiết lập lại khi phục dựng.
LDI-NET [40]	Feature Tokenizer và Token Encoder	Bộ giải mã đa nhãn dự đoán loại cây, loại bệnh, mức độ nghiêm trọng được thay bằng average pooling toàn cục và một lớp phân loại tuyến tính năm lớp.
InceptionV3 + SVM [35]	Backbone đóng băng để trích đặc trưng, SVM để phân loại	Thích nghi cho ChiliEdge-DS; áp dụng StandardScaler, PCA giữ 95% phương sai, và lựa chọn khám phá mô tả ở mục 4.1.
Meta-Ensemble [24]	MLP-Mixer, nhánh CNN-LSTM, bộ phân loại meta SVM	Đầu ra đổi thành năm lớp; cấu hình patch-embedding được cài đặt lại do mô tả gốc không cung cấp đầy đủ chi tiết.

Số lần chạy và cách kiểm định. Kết quả của mô hình đề xuất và các ablation là một lần chạy (seed 42); chuyên đề không báo cáo trung bình hay độ lệch chuẩn qua nhiều seed, và không diễn giải các kết quả này như bằng chứng về độ ổn định qua nhiều lần khởi tạo. Kiểm định McNemar được dùng để đánh giá chênh lệch giữa hai mô hình theo từng ảnh trên cùng 373 ảnh test. Các bảng ghi cặp số đếm b/c và giá trị p theo ký hiệu ở mục 1.7.2. Kiểm định McNemar không thay thế việc đánh giá phương sai qua nhiều seed; hạn chế này được bàn ở Chương 7. Trong chuyên đề này,

McNemar được áp dụng cho các so sánh giữa ChiliEdgeNet và các biến thể ablation trên cùng 373 ảnh test (Bảng 4.5–4.9), và cho phép so sánh ghép cặp giữa hai checkpoint trên cùng 1.463 ảnh truy vấn liên bộ dữ liệu (Bảng 4.13); kiểm định này không được dùng để xếp hạng toàn bộ 17 baseline ở Bảng 4.3.

Kiến trúc nền tảng RTR_Lite_MobileNet [7] ở mục 4.2 cũng được phục dựng với seed 42. Phép so sánh vì vậy khớp seed và theo cùng quy trình, nhưng mỗi kiến trúc vẫn chỉ được chạy một lần.

Cách đo độ trễ suy luận. Độ trễ CPU của ChiliEdgeNet là 20,24 ms/ảnh, tương đương 49,41 ảnh/giây, đo trên CPU Intel Xeon 2,00 GHz với 4 lõi logic trên một máy ảo Kaggle. Thiết lập đo dùng PyTorch FP32, batch size 1, với 10 vòng làm nóng và 50 vòng đo. Con số 49,41 ảnh/giây là thông lượng suy luận quy đổi từ một benchmark máy chủ đơn ảnh, không phải FPS của một ứng dụng camera. Trên cùng môi trường, GPU Tesla T4 đạt 5,46 ms/ảnh ở batch size 1 và 0,82 ms/ảnh ở batch size 32. CPU sử dụng là một CPU x86 hạng máy chủ, không phải phần cứng biên; các kết quả này vì vậy chỉ phản ánh chi phí suy luận trong môi trường thực nghiệm hiện tại.

4.2. So sánh với các mô hình SOTA

Ở seed 42, ChiliEdgeNet đạt 97,86% ở kịch bản baseline và 98,93% ở kịch bản augmented; macro-F1 tương ứng là 0,9790 và 0,9897. Tổng cộng 17 baseline từ 15 nghiên cứu gần đây được phục dựng và đánh giá dưới cùng khung thực nghiệm, trình bày đầy đủ ở Bảng 4.3. Vì mỗi baseline chỉ chạy một lần và không được tinh chỉnh riêng cho ChiliEdge-DS, bảng này phản ánh kết quả dưới cùng dữ liệu, cùng lần chia, cùng độ đo, không phải một xếp hạng tuyệt đối giữa các kiến trúc.

Bảng 4.3. Độ chính xác hai kịch bản, macro-F1 và số tham số trên ChiliEdge-DS.

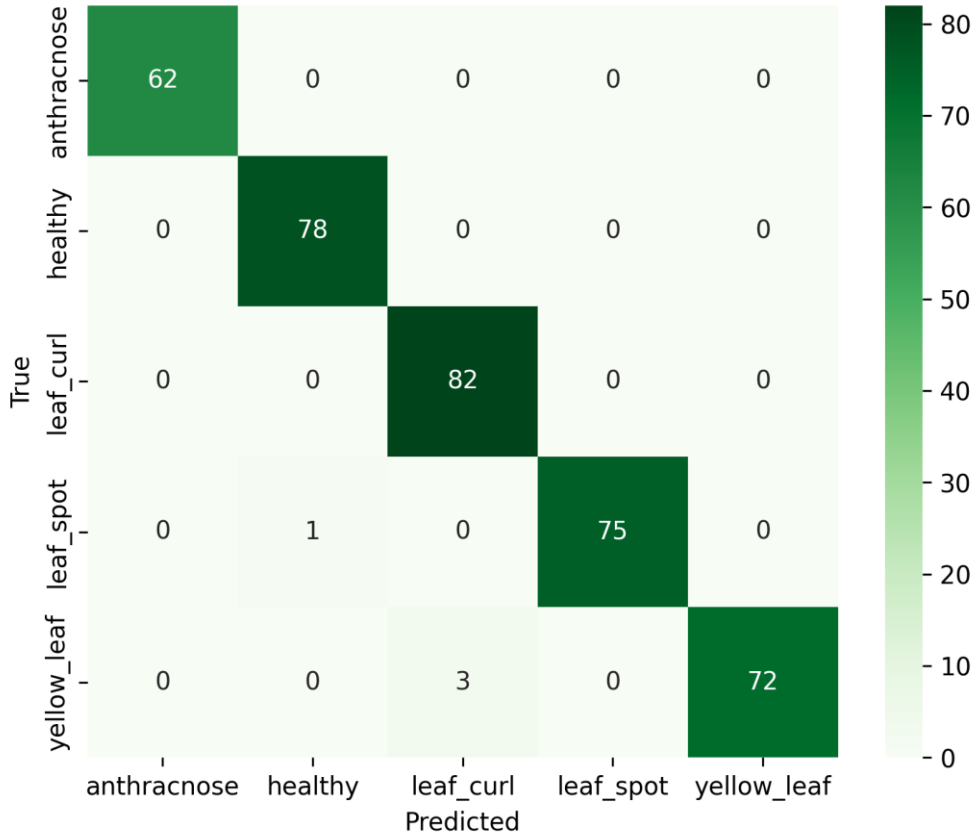
Mô hình	**Acc Baseline (%)**	**Acc Augmented (%)**	**F1-macro (Aug)**	**Tham số**
ChiliEdgeNet (đề xuất)	**97,86**	**98,93**	**0,9897**	**671.173**
RTR_Lite_MobileNet [7]	97,32	98,12	0,9821	1.032.568

Mô hình	Acc Baseline (%)	Acc Augmented (%)	F1-macro (Aug)	Tham số
GrapeLeafNet [22]	91,15	97,59	0,9769	5.308.035
PMVT-XS [23]	82,31	96,25	0,9644	2.367.515
ResNet-50 [27]	95,98	95,71	0,9586	23.597.957
EfficientNetB0 [27]	94,37	94,37	0,9463	4.055.976
TrIncNet [9]	80,70	90,62	0,9092	7.186.661
InceptionV3 + SVM [35]	93,83	90,35	0,9069	25.112.264
Meta-Ensemble [24]	88,74	88,74	0,8908	1.252.638
CustomMNetV3L [31]	87,67	86,86	0,8639	3.275.909
ULEN [37]	93,03	85,52	0,8603	28.069
MaizeFormerX [32]	79,89	82,04	0,8185	3.553.061
AlexNet [27]	85,52	77,75	0,7747	46.767.493
Lite-MDC [5]	68,10	76,68	0,7658	2.288.965
ViT [4]	72,39	73,99	0,7472	50.013.061
MRW-CNN [20]	74,26	73,73	0,7377	797.413
PDLC-ViT (chỉ phân loại; [15])	74,26	70,78	0,7054	17.547.269
LDI-NET [40]	70,78	68,63	0,6839	8.798.597

(Mỗi mô hình là một lần chạy seed 42 dưới cùng khung đánh giá; chênh lệch dưới khoảng 1 điểm phần trăm tương ứng chỉ 1–4 ảnh trên 373.)

So sánh với kiến trúc nền tảng. Ở lần chạy seed 42, ChiliEdgeNet đạt 98,93% ở kịch bản augmented, cao hơn RTR_Lite_MobileNet [7] (98,12%) 0,80 điểm phần trăm, tương đương 3 ảnh trên 373 ảnh test. ChiliEdgeNet dùng 671.173 tham số so với 1.032.568 tham số của RTR_Lite_MobileNet, giảm khoảng 35%. Cả hai mô hình dùng seed 42 và cùng quy trình, nhưng mỗi mô hình chỉ chạy một lần; hai khoảng tin cậy Wilson 95% chồng lấn nhau, nên chênh lệch độ chính xác cần được diễn giải thận trọng. Khoản giảm tham số thì không phụ thuộc vào tập test.

Phân tích lỗi. Ở kịch bản augmented, ChiliEdgeNet dự đoán sai một ảnh leaf_spot thành healthy và ba ảnh yellow_leaf thành leaf_curl (Hình 4.1). Độ chính xác 98,93% có khoảng tin cậy Wilson 95% là [97,28%; 99,58%]. Các lỗi còn lại tập trung ở ranh giới giữa triệu chứng úa vàng và xoăn lá, hai triệu chứng dễ nhầm lẫn ngay cả khi quan sát bằng mắt người.



Hình 4.1. Ma trận nhầm lẫn của ChiliEdgeNet trên tập test (kịch bản augmented).

4.3. Phân tích ablation cấp khối

Mọi biến thể được huấn luyện lại từ đầu với seed 42, cùng dữ liệu, cùng công thức huấn luyện (Bảng 4.1) và cùng 30 epoch; dòng "hiện tại" chính là mô hình đề xuất huấn luyện trong cùng quy trình. Mô hình hiện tại không chứa Triplet Attention lẫn RES_Block. Vì độ chính xác trên ảnh sạch đã gần bão hòa, vai trò của khối Tiny Transformer còn được khảo sát thêm bằng một stress test đa mức trên ảnh suy giảm tổng hợp ở mục 4.4.

Hai phép cắt trực tiếp lần lượt loại bỏ khối Tiny Transformer và vô hiệu hóa toàn bộ kết nối tắt của các khối InvertedResidual trong xương sống. Với Triplet Attention và RES_Block, chuyên đề dùng thiết kế thêm lại kiểu 2×2: cả hai cùng có mặt, chỉ Triplet, chỉ RES_Block, và không khối nào (mô hình hiện tại).

Bảng 4.4. Kết quả ablation kiến trúc trên ảnh sạch, seed 42.

Cấu hình	Tham số	Accuracy (%)
Hiện tại: không Triplet/RES_Block	671.173	98,93
Bỏ Tiny Transformer	555.845	99,20
Vô hiệu hóa kết nối tắt MobileNetV2	671.173	96,51
Thêm Triplet + RES_Block	671.732	97,59
Chỉ Triplet	671.473	98,12
Chỉ RES_Block	671.432	98,93

Bảng 4.5. Kiểm định McNemar cho hai phép cắt trực tiếp trên ảnh sạch.

Phép cắt	Delta (pp)	b / c	p	p_{Holm}
Bỏ Tiny Transformer	−0,27	0 / 1	1,0000	1,0000
Vô hiệu hóa kết nối tắt MobileNetV2	+2,41	10 / 1	0,0117	0,0234

(Delta dương nghĩa là mô hình hiện tại tốt hơn; p_{Holm} hiệu chỉnh cho hai phép so sánh định trước.)

Trên ảnh sạch, biến thể không có Tiny Transformer đạt cao hơn đúng một ảnh, nên lần chạy này không cho thấy lợi ích của khối này (Bảng 4.4). Ngược lại, vô hiệu hóa các kết nối tắt làm giảm chín ảnh dự đoán đúng ròng, và chênh lệch vẫn có ý nghĩa sau hiệu chỉnh Holm (Bảng 4.5). Kết quả này ủng hộ việc giữ các kết nối tắt của MobileNetV2. Đây cũng là kết quả ablation rõ nhất trong Chương 4.

Bảng 4.6. Ba phép so sánh loại bỏ Triplet Attention/RES_Block khỏi cấu hình có cả hai.

So sánh	Delta (pp)	p_{Holm}
Bỏ RES_Block	+0,54	0,625
Bỏ Triplet Attention	+1,34	0,375
Bỏ cả hai, về lại mô hình hiện tại	+1,34	0,375

(Giá trị *p* đã hiệu chỉnh Holm cho ba phép so sánh trên ảnh sạch.)

Không phép so sánh nào trong Bảng 4.6 đạt ý nghĩa thống kê sau hiệu chỉnh Holm. Ở lần chạy này, riêng RES_Block cho độ chính xác bằng mô hình hiện tại, trong khi Triplet Attention cho thấy xu hướng giảm hiệu năng. Các kết quả này ủng

hộ việc loại bỏ cả hai khối để tránh độ phức tạp không cần thiết, nhưng không chứng minh hai kiến trúc là tương đương.

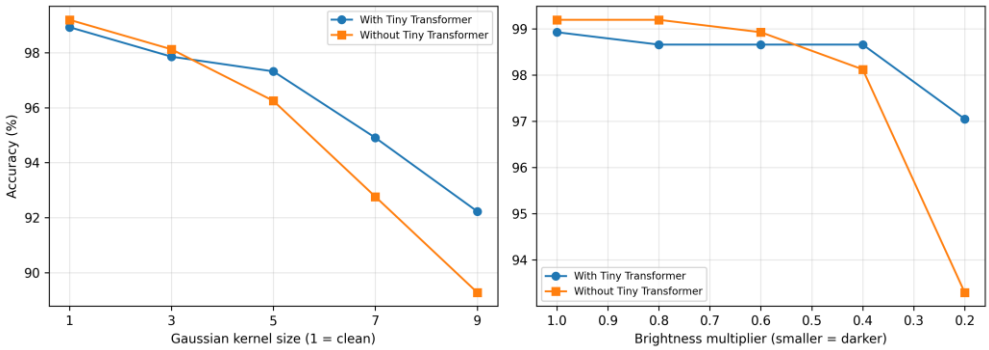
4.4. Stress test vai trò của khối Tiny Transformer trước suy giảm ảnh

Stress test dùng cùng 373 ảnh test và hai checkpoint seed 42: ChiliEdgeNet đầy đủ và một biến thể huấn luyện lại sau khi bỏ khối Tiny Transformer. Nhòe Gaussian được sinh bằng cv2.GaussianBlur (sigmaX=0) ở các kernel 3, 5, 7 và 9; giảm sáng được sinh bằng cách nhân toàn bộ cường độ pixel với hệ số 0,8, 0,6, 0,4 và 0,2. Tám điều kiện được cố định trước khi chấm điểm; ảnh sạch chỉ là điểm tham chiếu, không thuộc họ tám phép kiểm định. Kiểm định McNemar chính xác có hiệu chỉnh Holm được áp dụng trên cả tám điều kiện suy giảm cùng lúc. Kết quả của hai checkpoint ở từng mức suy giảm được tổng hợp ở Bảng 4.7 và Hình 4.2.

Bảng 4.7. Stress test khối Tiny Transformer dưới các mức suy giảm tổng hợp.

Điều kiện	**Có Transformer (%)**	**Không Transformer (%)**	**Delta (pp)**	**b / c**	p_{Holm}
Sạch	98,93	99,20	−0,27	0 / 1	-
Nhòe, kernel 3	97,86	98,12	−0,27	2 / 3	1,0000
Nhòe, kernel 5	97,32	96,25	+1,07	6 / 2	1,0000
Nhòe, kernel 7	94,91	92,76	+2,14	15 / 7	0,8028
Nhòe, kernel 9	92,23	89,28	+2,95	22 / 11	0,5610
Giảm sáng ×0,8	98,66	99,20	−0,54	0 / 2	1,0000
Giảm sáng ×0,6	98,66	98,93	−0,27	1 / 2	1,0000
Giảm sáng ×0,4	98,66	98,12	+0,54	3 / 1	1,0000
Giảm sáng ×0,2	97,05	93,30	+3,75	16 / 2	**0,0105**

(Delta dương nghĩa là mô hình có Tiny Transformer tốt hơn.)



Hình 4.2. Độ chính xác của ChiliEdgeNet theo mức suy giảm ảnh, có và không có khối Tiny Transformer.

Trên ảnh sạch và các mức suy giảm nhẹ, khối Tiny Transformer chưa cho thấy khác biệt rõ ràng. Từ kernel 5 trở đi, ưu thế dưới điều kiện nhòe tăng dần theo mức suy giảm (Hình 4.2), nhưng từng phép so sánh riêng lẻ không đạt ý nghĩa sau hiệu chỉnh Holm. Ở hệ số giảm sáng 0,2, mô hình đầy đủ đạt 97,05%, cao hơn 3,75 điểm phần trăm, giảm số lỗi từ 25 xuống 11 ảnh (phép so sánh duy nhất còn ý nghĩa sau hiệu chỉnh Holm, $p_{Holm} = 0{,}0105$). Các kết quả này cho thấy khối Tiny Transformer giúp duy trì biểu diễn đặc trưng dưới điều kiện suy giảm thị giác nghiêm trọng, hơn là mang lại một lợi thế ổn định trên ảnh sạch. Lợi ích này đi kèm chi phí 115.328 tham số bổ sung so với biến thể không có Transformer.

4.5. Ablation cấp từng kỹ thuật

Mỗi biến thể thay đúng một kỹ thuật trong khối Tiny Transformer bằng phương án tiêu chuẩn tương ứng, giữ nguyên các thành phần còn lại.

Bảng 4.8. Ablation từng kỹ thuật trong khối Tiny Transformer trên ảnh sạch, seed 42.

Biến thể	**Thay thế**	**Tham số**	**Accuracy (%)**
ChiliEdgeNet	2D-RoPE + GQA + RMSNorm	671.173	98,93
learned_pe	2D-RoPE → PE học được	696.261	98,93
mha	GQA → Multi-Head Attention	687.557	98,93
layernorm	RMSNorm → LayerNorm	671.557	98,39

Trên ảnh sạch (Bảng 4.8), PE học được và MHA cho đúng cùng tập ảnh đúng/sai với ChiliEdgeNet; LayerNorm thấp hơn hai ảnh, nhưng McNemar không cho thấy ý nghĩa thống kê ($p = 0,5$). Dữ liệu một lần chạy này chưa cho thấy ba lựa chọn (2D-RoPE, GQA và RMSNorm) vượt trội về độ chính xác. Về cấu trúc tham số, ba kỹ thuật lần lượt tiết kiệm 25.088, 16.384 và 384 tham số so với phương án tiêu chuẩn, tổng cộng 41.856 tham số nếu dùng đồng thời cả ba phương án tiêu chuẩn thay thế; biến thể thay cả ba cùng lúc chưa được huấn luyện riêng để kiểm chứng.

4.6. Đánh giá liên bộ dữ liệu sau thích nghi few-shot

Các stress test ở mục 4.4 chỉ tổng hợp nhòe và giảm sáng từ chính ảnh ChiliEdge-DS, nên chưa nắm bắt đầy đủ khoảng cách miền thu nhận ảnh (acquisition domain gap). Chuyên đề vì vậy bổ sung một đánh giá trên bộ dữ liệu Chili Plant Leaf Disease and Growth Stage Dataset thu thập tại Bangladesh, phiên bản 1, công bố trên Mendeley Data theo giấy phép CC BY 4.0 [29]. Ảnh nguồn được thu thập tại một khu vườn ở Charpolisha, Jamalpur (Bangladesh) từ tháng 10/2024 đến tháng 2/2025; phần ảnh lá bệnh gồm 1.856 ảnh thuộc sáu nhãn nguồn. Động cơ của thí nghiệm giống các nghiên cứu ở mục 2.7.

Chỉ bốn nhãn có tương ứng trực tiếp với không gian nhãn của ChiliEdgeNet được sử dụng: Healthy Leaf → healthy, Curl Virus → leaf_curl, Cercospora Leaf Spot → leaf_spot, Nutrition Deficiency → yellow_leaf. Hai nhãn Bacterial Spot (156 ảnh) và White spot (195 ảnh) bị loại vì mô hình không có lớp đích tương ứng. Bộ dữ liệu nguồn không có lớp anthracnose, nên không có hàng ground-truth cho lớp này; tuy vậy checkpoint vẫn giữ nguyên cả năm đầu ra đã huấn luyện, và mọi dự đoán nghiêng về anthracnose vẫn được tính là sai, cột dự đoán này vẫn được giữ trong ma trận nhầm lẫn.

Thiết lập few-shot. Tập hỗ trợ gồm 42 ảnh lấy nguyên trạng từ Nirob và cộng sự [29]: 8 healthy, 9 leaf_curl, 16 leaf_spot, 9 yellow_leaf (Bảng 4.9). Các ảnh hỗ trợ được thêm vào tập huấn luyện của ChiliEdge-DS để tạo một checkpoint thích nghi riêng; tập val/test trong miền của ChiliEdge-DS giữ nguyên không đổi. Đây là thích

nghi miền có giám sát bằng một tập hỗ trợ nhỏ, không phải học meta theo từng episode (episodic meta-learning). Số lượng ảnh hỗ trợ khác nhau giữa các lớp, nên đây cũng không phải một thiết lập N-way K-shot cân bằng.

Để kiểm soát rò rỉ dữ liệu, tập hỗ trợ và tập truy vấn được theo dõi bằng băm SHA-256 thay vì chỉ dựa vào tên file. Tập truy vấn được định nghĩa là phần bù của 42 ảnh hỗ trợ trong tổng số 1.505 ảnh thuộc bốn lớp, và một cuộc kiểm tra xác nhận giao của hai tập bằng không. Checkpoint few-shot được huấn luyện với seed 42.

Bảng 4.9. Phân chia tập hỗ trợ/truy vấn đã xác minh từ bốn lớp được ánh xạ.

Nhãn nguồn (Nirob và cộng sự [29])	**Nhãn ChiliEdgeNet**	**Ảnh gốc**	**Hỗ trợ few-shot**	**Truy vấn sạch**	**Thư mục test đã hiệu chỉnh**
Healthy Leaf	healthy	458	8	450	450
Curl Virus	leaf_curl	423	9	414	414
Cercospora Leaf Spot	leaf_spot	180	16	164	164
Nutrition Deficiency	yellow_leaf	444	9	435	435
Tổng		**1.505**	**42**	**1.463**	**1.463**

Để hỗ trợ tái lập độc lập, tập truy vấn sạch 1.463 ảnh được công bố dưới tên Chili Leaf Disease Cross-Dataset Query Set, Kaggle phiên bản 2 [11], và 42 ảnh hỗ trợ được công bố dưới tên Chili Leaf Disease Few-Shot, Kaggle phiên bản 1 [12]. Cả hai gói đều là tập con dẫn xuất từ bộ dữ liệu Mendeley Data gốc và không thay thế yêu cầu trích dẫn bộ dữ liệu nguồn [29].

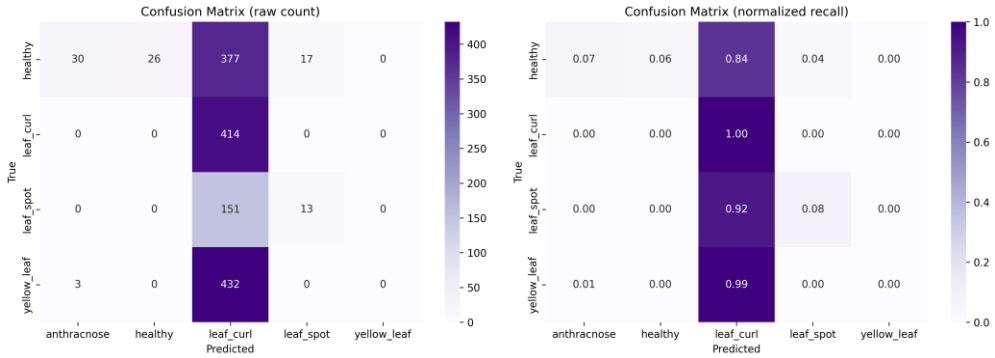
Kết quả trên tập truy vấn sạch. Cả hai checkpoint được đánh giá trên cùng tập truy vấn sạch 1.463 ảnh (Bảng 4.10). Checkpoint gốc dùng seed 42 (đã xác minh SHA-256); checkpoint sau thích nghi few-shot cũng dùng seed 42 (đã xác minh SHA-256). Ảnh được resize về 224×224 pixel và chuẩn hóa theo thống kê ImageNet. Precision, recall và F1 trung bình vĩ mô chỉ tính trung bình trên bốn lớp có ground-truth; các dự đoán nghiêng về anthracnose vẫn làm giảm độ chính xác.

Bảng 4.10. Kết quả trước và sau thích nghi few-shot trên tập truy vấn sạch 1.463 ảnh.

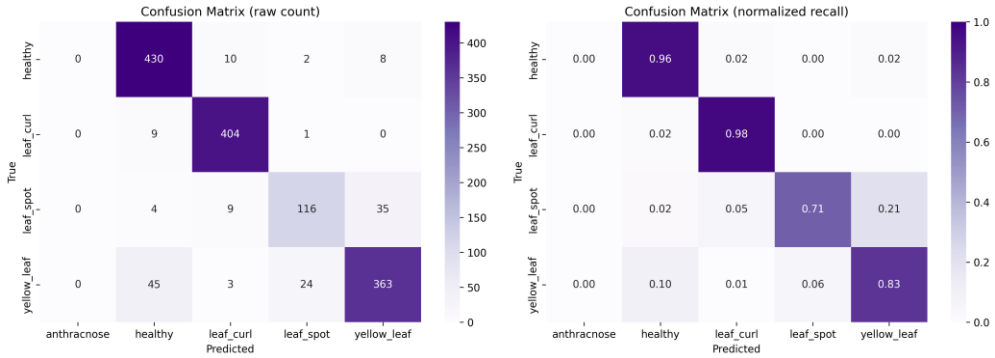
Cấu hình	**Seed**	**Accuracy**	**Precision-macro**	**Recall-macro**	**F1-macro**	**Số ảnh đúng**

Checkpoint gốc, chưa few-shot	42	30,96%	0,4337	0,2843	0,1766	453/1.463
Checkpoint sau few-shot	42	89,75%	0,8837	0,8683	0,8744	1.313/1.463
Chênh lệch		**+58,78 pp**	**+0,4500**	**+0,5840**	**+0,6978**	**+860**

Ở checkpoint gốc, leaf_curl có recall 1,0000 nhưng precision chỉ 0,3013, trong khi yellow_leaf có recall bằng không (Hình 4.3); mô hình gần như "đổ dồn" mọi dự đoán về leaf_curl khi gặp ảnh ngoài miền huấn luyện. Sau thích nghi few-shot, F1 của healthy, leaf_curl, leaf_spot và yellow_leaf lần lượt đạt 0,9168, 0,9619, 0,7557 và 0,8633 (Hình 4.4). So sánh theo từng ảnh cho thấy 443 ảnh được cả hai checkpoint dự đoán đúng, 140 ảnh cả hai đều sai, 870 ảnh chuyển từ sai ở checkpoint gốc sang đúng sau few-shot, và chỉ 10 ảnh theo chiều ngược lại. Kiểm định McNemar chính xác hai phía cho $p = 1{,}83 \times 10^{-242}$. Đây là một phép so sánh ghép cặp đơn lẻ trên một bộ dữ liệu ngoài miền, không thuộc họ kiểm định Holm của phần ablation.



Hình 4.3. Ma trận nhầm lẫn của checkpoint gốc trên tập truy vấn sạch 1.463 ảnh.



Hình 4.4. Ma trận nhầm lẫn của checkpoint sau thích nghi few-shot trên cùng tập truy vấn sạch 1.463 ảnh.

4.7. Đo độ trễ tiền xử lý và suy luận trên ứng dụng camera laptop

ChiliEdgeNet được tích hợp vào một ứng dụng camera laptop, nơi người dùng đặt một chiếc lá vào một vùng quan tâm (Region of Interest, ROI) và kích hoạt dự đoán trên từng khung hình. Thí nghiệm này chỉ đánh giá độ trễ tiền xử lý cộng suy luận, như một minh chứng khả thi cho việc tích hợp; nó không dùng khung hình camera để ước lượng độ chính xác trong điều kiện thực địa.

Phiên đo gồm 62 lượt dự đoán, chạy ở batch size 1 trên CPU Intel Core i7-13650HX thế hệ 13. Phép đo gồm tiền xử lý, suy luận, softmax và chọn lớp top-1; giao thức chi tiết ở mục 6.4.2. Bảng 4.11 tổng hợp các thống kê độ trễ của phiên đo này.

Bảng 4.11. Độ trễ tiền xử lý và suy luận của ChiliEdgeNet trên CPU laptop.

Chỉ số	**Độ trễ (ms)**
Số lượt đo	62
Trung bình	32,18
Trung vị	29,70
Bách phân vị 95	46,49
Nhỏ nhất	17,33
Lớn nhất	122,76

Trung vị 29,70 ms và bách phân vị 95 là 46,49 ms cho thấy phần tiền xử lý cộng suy luận phản hồi nhanh trên CPU laptop mà không cần GPU. Kết quả này chỉ đến từ một phiên đo trên một máy và chưa tính toàn bộ chi phí từ lúc chụp đến lúc hiển thị. Máy thử cũng không phải phần cứng biên chuyên dụng, nên đây mới là bằng chứng sơ bộ về khả năng tích hợp. Độ chính xác trong phiên này không được báo cáo vì dữ liệu gán nhãn không cân bằng và có nhiều khung hình lặp của cùng một lá.

Mô tả chi tiết chương trình demo và giao thức đo được trình bày ở mục 6.4.

4.8. Khảo sát mở rộng về nén mô hình

Mô hình đề xuất đã rất nhẹ (671.173 tham số, khoảng 2,7 MB trọng số ở FP32; tệp checkpoint trên đĩa 2,838 MB) so với kiến trúc nền tảng và toàn bộ nhóm SOTA phục dựng (Bảng 4.3). Mục này kiểm chứng bằng thực nghiệm liệu có thể nén thêm nữa cho mục tiêu triển khai thiết bị biên hay không, bằng hai kỹ thuật nén chuẩn: cắt tỉa (pruning) và lượng tử hóa (quantization).

Cắt tỉa không cấu trúc (unstructured pruning). Áp dụng cắt tỉa toàn cục theo độ lớn trọng số (global magnitude pruning, chuẩn L1) ở ba mức thưa 20%, 40% và 60%, mỗi mức tinh chỉnh lại (fine-tune) trước khi đánh giá. Bảng 4.12 trình bày số tham số khác không và độ chính xác ở từng mức thưa.

Bảng 4.12. Đường cong độ thưa–độ chính xác của cắt tỉa không cấu trúc.

Độ thưa	Tham số khác không	Accuracy	F1-macro	Δ Accuracy
20%	540.370	98,66	0,9871	−0,27 pp
40%	**409.567**	**98,93**	**0,9897**	**0,00 pp**
60%	278.763	97,05	0,9720	−1,88 pp

Ở mức 40%, độ chính xác giữ nguyên so với mô hình gốc; ở 60%, mô hình bắt đầu mất độ chính xác đo được. Tuy nhiên cắt tỉa không cấu trúc chỉ đưa các trọng số bị cắt về giá trị 0, không thay đổi cách lưu trữ tensor, nên dung lượng file trên đĩa không giảm (Bảng 4.13: mọi biến thể cắt tỉa vẫn khoảng 2,838 MB, bằng bản gốc). Muốn thực sự giảm dung lượng từ cắt tỉa cần định dạng lưu trữ thưa chuyên dụng hoặc cắt tỉa có cấu trúc (loại bỏ nguyên kênh/khối), nằm ngoài phạm vi khảo sát này.

Lượng tử hóa động INT8 và FP16. Đo thêm lượng tử hóa động sau huấn luyện (dynamic post-training quantization) xuống INT8 cho các lớp Linear, và chuyển toàn bộ mô hình sang FP16 chạy trên GPU.

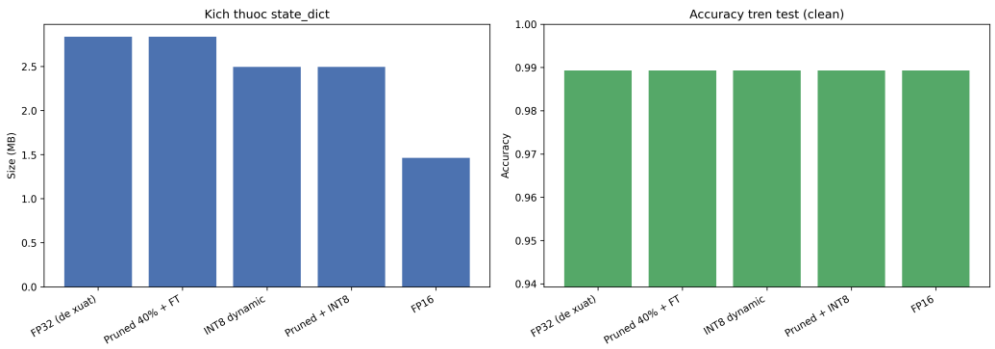
Bảng 4.13. So sánh các phương án nén (đo trên cùng 373 ảnh test, kịch bản augmented, phiên đo riêng).

Biến thể	Dung lượng	Độ thưa	Accuracy	F1-macro	Độ trễ CPU	Độ trễ GPU

FP32 (đề xuất)	2,838 MB	0%	98,93	0,9897	21,37 ms	5,23 ms
Cắt tỉa 40% + tinh chỉnh	2,838 MB	40%	98,93	0,9897	20,91 ms	5,51 ms
INT8 động	2,494 MB	0%	98,93	0,9897	22,13 ms	-
Cắt tỉa + INT8	2,494 MB	40,2%	98,93	0,9897	21,52 ms	-
FP16	**1,462 MB**	0%	98,93	0,9897	-	6,27 ms

Không kỹ thuật nào cải thiện đáng kể so với FP32 gốc trên phần cứng máy chủ ảo đã thử, ngoại trừ dung lượng của FP16. Ba quan sát đáng chú ý: cắt tỉa không cấu trúc không giảm dung lượng thực tế dù độ thưa "trên giấy" trông ấn tượng; INT8 động chỉ giảm khoảng 12% dung lượng (2,838 → 2,494 MB) nhưng lại chậm hơn FP32 trên CPU của phép đo này (22,13 ms so với 21,37 ms), có thể vì mô hình đã đủ nhỏ nên chi phí chuyển đổi kiểu dữ liệu động lúc suy luận vượt quá lợi ích tiết kiệm băng thông bộ nhớ; và FP16 nhỏ nhất (1,462 MB, giảm 48,5% so với FP32) nhưng chỉ đo được trên GPU (trong phép thử này, PyTorch trên CPU không chạy FP16 hiệu quả cho kiến trúc này) và độ trễ GPU cũng nhỉnh hơn FP32.

Accuracy và F1-macro giữ nguyên tuyệt đối qua mọi biến thể ở độ thưa 40% trở xuống; cắt tỉa/lượng tử hóa ở mức vừa phải không đánh đổi độ chính xác trên bộ dữ liệu này, nhưng cũng không mang lại lợi ích tốc độ hay dung lượng rõ rệt ngoài FP16. Kết luận thực dụng của khảo sát mở rộng này: ở quy mô 671.173 tham số, mô hình đã đủ nhỏ để triển khai trực tiếp bằng FP32 trên phần cứng đã thử; nén sâu hơn chỉ có ý nghĩa khi đo trên phần cứng biên vật lý có nhân tính toán INT8/FP16 chuyên dụng (ARM NEON, NPU di động) thay vì CPU/GPU máy chủ ảo, đúng như hướng phát triển đã nêu ở mục 7.2. Hình 4.5 tổng hợp trực quan các phương án nén nói trên.



Hình 4.5. Tổng hợp trực quan các phương án nén mô hình.

Các kết quả định lượng trong chương này cho biết mô hình đạt được gì trên dữ liệu thử nghiệm. Chương 5 tiếp theo xem xét mô hình dựa vào những vùng nào của ảnh để đưa ra quyết định, thông qua ba kỹ thuật XAI và một đối chiếu với mô hình đa phương thức tổng quát.

Chương 5. DIỄN GIẢI MÔ HÌNH

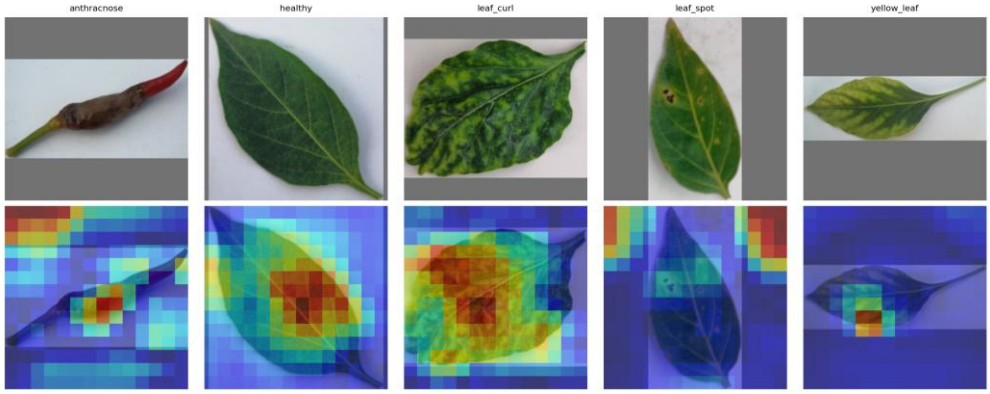
Chương này xem xét các quyết định của ChiliEdgeNet từ góc độ diễn giải. Mục 5.1 dùng ba kỹ thuật XAI thuộc ba nhóm cơ chế khác nhau để kiểm tra mô hình tập trung vào vùng nào của ảnh; mục 5.2 đối chiếu mô hình với Gemini 3.1 Flash Lite, một mô hình đa phương thức tổng quát chạy ở chế độ zero-shot.

5.1. AI giải thích được (XAI)

Các phân tích trong chương này dùng checkpoint gốc của ChiliEdgeNet, checkpoint được huấn luyện chỉ trên ChiliEdge-DS và dùng để tạo ra kết quả trong miền ở Bảng 4.3 (kịch bản augmented, seed 42). Checkpoint sau thích nghi few-shot dùng cho thí nghiệm chuyển miền ở mục 4.6 là một mô hình thích nghi riêng, không được dùng để tạo các kết quả diễn giải dưới đây.

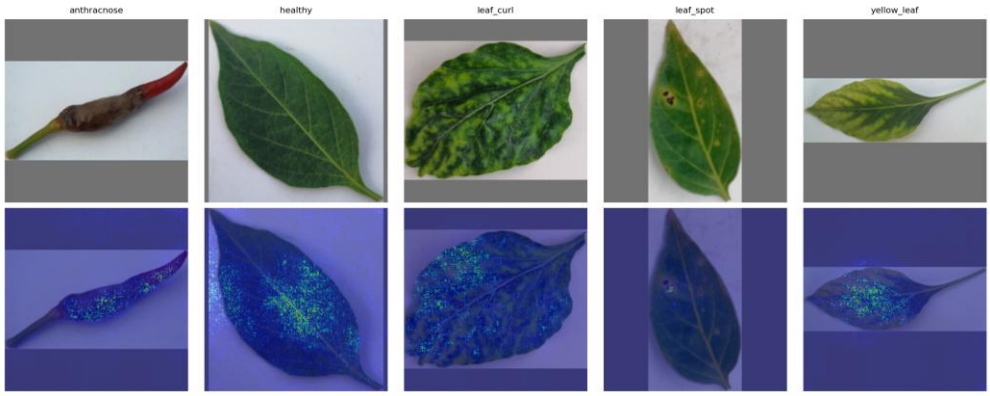
Như đã lập luận ở mục 1.6, độ chính xác cao chưa đủ để tin cậy một mô hình, đặc biệt khi phần lớn ảnh trong ChiliEdge-DS đều chụp trên nền trắng có kiểm soát, một manh mối giả tiềm tàng mà mô hình có thể bám vào thay vì đặc trưng bệnh thật sự. Chuyên đề áp dụng ba kỹ thuật XAI bổ trợ lẫn nhau trên checkpoint gốc, mỗi kỹ thuật dùng một ảnh đại diện cho mỗi lớp trong năm lớp:

- Occlusion Sensitivity che lần lượt từng vùng 32×32 pixel rồi đo mức thay đổi đầu ra, cho thấy độ nhạy ở cấp vùng ảnh (Hình 5.1).
- Gradient SHAP (phương pháp Expected Gradients [8]) ước lượng đóng góp từng pixel từ gradient, tính với 20 mẫu và hai tensor baseline hằng số (0 và 1) trong không gian đầu vào đã chuẩn hóa, cho phân bố đóng góp ở cấp pixel trên bề mặt đối tượng (Hình 5.2).
- Attention Rollout tổng hợp dòng chú ý qua khối Transformer trên toàn bộ lưới 14×14; vì kiến trúc dùng mean pooling và không có token CLS, bản đồ Rollout được tạo bằng cách lấy trung bình dòng chú ý trên 196 token, phản ánh cách lan truyền chú ý nội tại của khối Transformer (Hình 5.3).



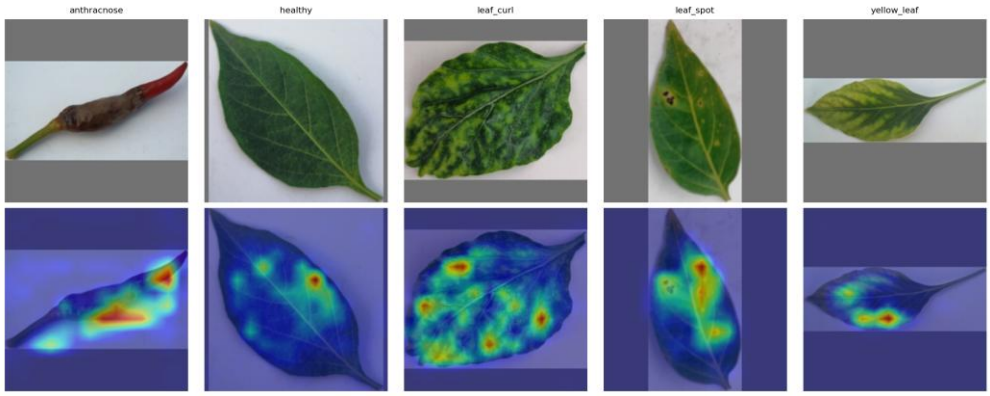
Hình 5.1. Occlusion Sensitivity cho năm lớp bệnh.

Hàng trên là ảnh gốc, hàng dưới là bản đồ độ nhạy Occlusion Sensitivity chồng lên; vùng sáng là nơi che đi làm thay đổi nhiều nhất đầu ra của mô hình, kỹ thuật này coi mô hình như hộp đen.



Hình 5.2. Gradient SHAP cho năm lớp.

Hàng trên là ảnh gốc, hàng dưới là độ lớn đóng góp theo pixel của Gradient SHAP chồng lên.



Hình 5.3. Attention Rollout của khối Tiny Transformer.

Bản đồ được tổng hợp trên lưới 14×14 theo cách mô tả ở mục 5.1.

Ba bản đồ mang lại ba góc nhìn khác nhau: Occlusion cho thấy độ nhạy ở cấp vùng, Gradient SHAP cho phân bố đóng góp ở cấp pixel trên bề mặt đối tượng, còn Attention Rollout phản ánh sự lan truyền chú ý nội tại của Transformer. Trên năm ảnh minh họa, các vùng phản hồi cao nhìn chung giao với vùng lá/quả hoặc vùng bệnh, dù mức độ tập trung khác nhau giữa các kỹ thuật và giữa các lớp. Vì đánh giá chỉ dùng một ảnh cho mỗi lớp và không có phân vùng triệu chứng ground-truth để đối chiếu, đây là một khảo sát định tính mang tính khám phá; kết quả này chưa đủ để kết luận mô hình đã học đúng đặc trưng bệnh lý.

5.2. Đối chiếu với một mô hình đa phương thức tổng quát

Chuyên đề dùng mô hình đa phương thức tổng quát Gemini 3.1 Flash Lite, mã định danh API gemini-3.1-flash-lite, truy cập ngày 8/7/2026 [10], với cấu hình sinh mặc định, để phân loại trực tiếp ảnh gốc ở chế độ zero-shot. Mô hình được yêu cầu, bằng tiếng Việt, phân loại mỗi ảnh vào đúng một trong năm nhãn của ChiliEdge-DS; trường predicted_class được chấm điểm trực tiếp so với năm chuỗi nhãn này, không dịch từ đồng nghĩa hay ánh xạ hậu kiểm. Cả 373 lượt gọi API đều trả về kết quả hợp lệ, không có trường hợp bị loại. Đây là một đối chứng tham khảo thứ cấp, không phải đóng góp chính của chuyên đề: Gemini không được tinh chỉnh trên ChiliEdge-DS, trong khi ChiliEdgeNet là một bộ phân loại chuyên biệt được huấn luyện có giám sát cho đúng năm lớp.

Bảng 5.1. So sánh ChiliEdgeNet với Gemini 3.1 Flash Lite trên 373 ảnh test.

Hệ thống	**Accuracy**	**Độ trễ/ảnh**
ChiliEdgeNet (CPU cục bộ)	**98,93%**	**20,24 ms**
Gemini 3.1 Flash Lite	78,55%	2.123 ms (trung vị round-trip API)

Như tóm tắt ở Bảng 5.1, Gemini đạt 78,55% độ chính xác, còn ChiliEdgeNet đạt 98,93% trên cùng 373 ảnh test, khớp với Bảng 4.3. Trong ba ảnh yellow_leaf mà

ChiliEdgeNet dự đoán sai thành leaf_curl (mục 4.2), Gemini phân loại đúng cả ba thành yellow_leaf.

Về độ trễ. Con số khoảng 105 lần có được khi đặt 20,24 ms suy luận cục bộ cạnh 2.123 ms trung vị thời gian khứ hồi khi gọi API. Thời gian khứ hồi gồm cả độ trễ mạng, hàng đợi và cách nhà cung cấp gộp lô phía máy chủ. Vì vậy con số này phản ánh hai cách triển khai khác nhau, chưa nói lên tốc độ tính toán của hai kiến trúc. Khoảng cách độ chính xác cũng cần đọc tương tự: Gemini chạy zero-shot, còn ChiliEdgeNet được huấn luyện có giám sát cho đúng bài toán này. Endpoint thương mại có thể thay đổi theo thời gian, nên kết quả chỉ tái lập được với đúng mã mô hình, câu lệnh gợi ý và ngày truy cập đã nêu.

Chương 6. TRIỂN KHAI ỨNG DỤNG MINH HỌA

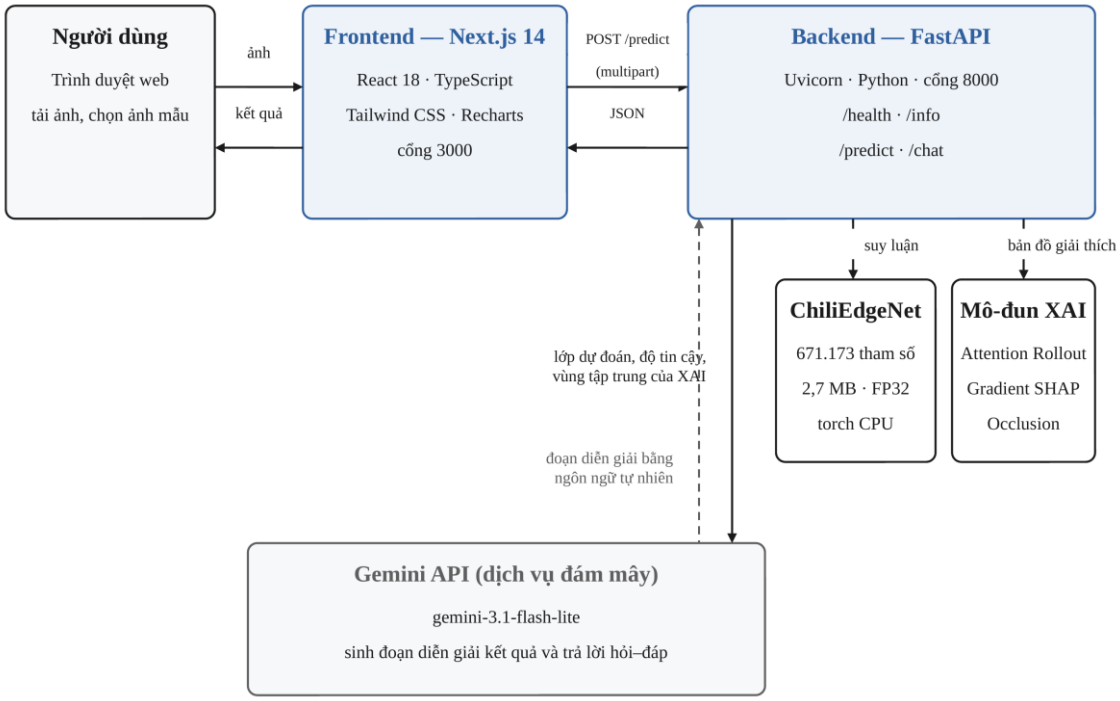
Chương này mô tả ứng dụng minh họa được xây dựng quanh ChiliEdgeNet. Ứng dụng gồm backend suy luận (FastAPI, Python) phục vụ mô hình ở định dạng FP32 và frontend (Next.js, React) hiển thị kết quả, các bản đồ diễn giải XAI (Chương 5) cùng một khung hỏi đáp dùng LLM. Hai thành phần giao tiếp qua REST API cục bộ. Trong chương này, chỉ phần tích hợp camera và đo độ trễ (mục 6.4) có số liệu định lượng, đã báo cáo ở mục 4.7. Giao diện web, khối XAI và khung hỏi đáp là phần minh họa sản phẩm, chưa được đánh giá bằng thực nghiệm.

6.1. Kiến trúc tổng thể ứng dụng

Backend nạp checkpoint ChiliEdgeNet đã huấn luyện (kịch bản augmented, mục 4.1) cùng các mô-đun XAI (Occlusion Sensitivity, Gradient SHAP, Attention Rollout, Chương 5) và lộ ra các endpoint REST cho dự đoán, diễn giải và kiểm tra độ bền. Frontend gọi các endpoint này để hiển thị kết quả theo thời gian gần thực, không cần cài đặt cục bộ ngoài trình duyệt. Backend chạy được trên CPU tiêu dùng (mục 4.1, 4.7).

6.1.1. *Tổ chức thành phần và luồng dữ liệu*

Ứng dụng được tách thành ba phần độc lập, mỗi phần nằm trong một thư mục riêng và có thể chạy tách rời: thư mục model chứa định nghĩa kiến trúc ChiliEdgeNet cùng checkpoint đã huấn luyện, thư mục backend là một dịch vụ REST viết bằng FastAPI nạp checkpoint đó và phục vụ suy luận, thư mục frontend là ứng dụng Next.js chạy trong trình duyệt. Cách tách này giữ cho phần mô hình không phụ thuộc vào tầng web: cùng một thư mục model được backend và chương trình demo camera ở mục 6.4 dùng chung, nên khi đổi checkpoint chỉ cần thay một tệp duy nhất, không phải sửa mã nguồn ở hai nơi và cũng không sợ lệch bước tiền xử lý giữa hai chương trình.



Hình 6.1. Kiến trúc tổng thể của ứng dụng minh họa.

Hình 6.1 mô tả luồng dữ liệu của một lượt chẩn đoán: trình duyệt gọi frontend Next.js, frontend gọi backend FastAPI qua REST, backend chạy ChiliEdgeNet cùng ba mô-đun XAI trên máy cục bộ và chỉ gửi phần kết quả dạng văn bản sang Gemini API để sinh đoạn diễn giải. Người dùng chọn ảnh trên trình duyệt; frontend đóng ảnh thành một yêu cầu multipart và gửi tới endpoint /predict của backend. Backend giải mã ảnh, đưa qua đúng chuỗi tiền xử lý đã dùng khi huấn luyện: đổi kích thước về 224×224, chuyển sang tensor, chuẩn hóa theo trung bình và độ lệch chuẩn của ImageNet, rồi chạy suy luận và lần lượt gọi ba mô-đun XAI trên cùng tensor đầu vào đó. Toàn bộ kết quả gồm lớp dự đoán, vector xác suất của năm lớp, ba bản đồ giải thích đã mã hóa base64, bản đồ đồng thuận và đoạn diễn giải được gói trong một đối tượng JSON duy nhất trả về frontend. Trình duyệt chỉ hiển thị, không thực hiện bất kỳ phép tính nào của mô hình.

6.1.2. *Backend suy luận*

Backend chạy trên Uvicorn ở cổng 8000, cung cấp bốn endpoint chia làm hai nhóm. Hai endpoint đầu phục vụ giao diện: /health cho biết dịch vụ đang chạy bình

thường hay không, kèm danh sách lớp và trạng thái cấu hình khóa API của mô hình ngôn ngữ; /info trả về thẻ thông tin hiển thị ở đầu trang, gồm số tham số, dung lượng checkpoint, độ chính xác trên tập test, thiết bị đang chạy và tốc độ suy luận. Hai endpoint còn lại mới là phần việc chính: /predict nhận ảnh và trả về chẩn đoán kèm giải thích, còn /chat phục vụ khung hỏi đáp ở mục 6.1.4.

Checkpoint được nạp một lần khi khởi động ứng dụng, nên các yêu cầu sau không mất thêm thời gian nạp mô hình.

Ngoài nhãn dự đoán, backend còn tính thêm hai cờ cảnh báo dựa trên chính phân bố xác suất đầu ra. Cờ uncertain bật lên khi khoảng cách xác suất giữa lớp cao nhất và lớp thứ hai nhỏ hơn 0,15, hoặc khi xác suất lớp cao nhất chưa tới 0,60, tức mô hình còn phân vân giữa hai khả năng. Cờ ood bật lên khi xác suất lớp cao nhất dưới 0,45, gợi ý ảnh đưa vào có thể không thuộc năm lớp đã huấn luyện. Hai ngưỡng này chọn theo kinh nghiệm, chỉ nhằm tránh việc giao diện khẳng định chắc chắn một nhãn trên ảnh lạ, không phải kết quả của một quy trình hiệu chuẩn xác suất hay một phép kiểm định ngoài phân bố nào, vì vậy chúng không nằm trong các con số đã đo ở Chương 4.

/predict còn nhận thêm ba tham số blur, dark, noise để làm suy giảm ảnh ngay trên máy chủ trước khi suy luận. Đây chính là cơ chế đứng sau khối kiểm tra độ bền ở mục 6.3, tái hiện đúng kiểu suy giảm tổng hợp đã dùng trong stress test ở mục 4.4, nhờ đó người xem có thể tự kiểm chứng bằng một thao tác kéo thanh trượt đơn giản điều mà Chương 4 vốn phải đo bằng số.

6.1.3. *Frontend*

Frontend viết bằng Next.js 14 với React 18 và TypeScript, dùng Tailwind CSS cho giao diện và thư viện Recharts cho biểu đồ xếp hạng xác suất. Toàn bộ ứng dụng gói gọn trong một trang: thẻ thông tin mô hình, khung kéo thả ảnh kèm thư viện ảnh mẫu chạy ngay khi bấm, khối kết quả, khối bản đồ giải thích có thanh trượt chỉnh độ đậm lớp phủ, khối kiểm tra độ bền và khối hỏi đáp. Địa chỉ backend đọc từ biến môi

trường NEXT_PUBLIC_API_URL nên khi triển khai lên tên miền thật chỉ cần đổi biến này rồi dựng lại, không phải sửa mã nguồn.

Khi người dùng kéo hai thanh trượt độ mờ và độ sáng ở khối kiểm tra độ bền, frontend không gọi lại toàn bộ /predict mà gửi một yêu cầu rút gọn có tắt cả ba mô-đun XAI, đồng thời gom các thao tác kéo liên tiếp bằng một khoảng chờ ngắn trước khi gửi. Nếu không làm vậy, mỗi lần nhích thanh trượt sẽ kéo theo một lượt tính ba bản đồ giải thích (phần tốn thời gian nhất của một yêu cầu) và giao diện sẽ giật.

6.1.4. *Sinh diễn giải kết quả bằng Gemini API*

Một mô hình phân loại chỉ trả ra một nhãn và một vector xác suất. Với người dùng cuối là nông dân, đó chưa phải thông tin dùng được ngay: họ cần biết bệnh đó là gì, mô hình có đang chắc chắn hay không, và nên xử lý thế nào. Chuyên đề dùng một mô hình ngôn ngữ lớn thông qua Gemini API (mã định danh gemini-3.1-flash-lite, cùng mô hình đã dùng để đối chiếu ở mục 5.2) để chuyển kết quả số thành lời.

Mô hình ngôn ngữ không nhìn thấy ảnh. Nó chỉ nhận phần văn bản mà backend đã tính xong: nhãn dự đoán, độ tin cậy, thứ hạng xác suất của cả năm lớp, và với mỗi kỹ thuật XAI là một chuỗi mô tả vùng ảnh mà kỹ thuật đó làm nổi bật cùng mức độ tập trung, ví dụ "trung tâm ảnh, tập trung cao". Chuỗi mô tả này do một hàm phía backend sinh ra bằng cách tìm khối trọng số lớn nhất trên bản đồ nhiệt rồi quy về chín vùng của khung ảnh, hoàn toàn không dùng đến mô hình ngôn ngữ. Việc phân loại do ChiliEdgeNet đảm nhận hoàn toàn trên máy cục bộ; mô hình ngôn ngữ chỉ diễn đạt lại kết quả.

Prompt yêu cầu mô hình viết một đoạn 150–250 chữ tiếng Việt gồm bốn ý: bệnh được dự đoán là gì và triệu chứng thường gặp; mô hình đang chắc chắn hay còn phân vân, căn cứ vào khoảng cách xác suất giữa lớp nhất và lớp nhì; các kỹ thuật XAI có đồng thuận về vùng ảnh hay không; và khuyến nghị xử lý ngắn gọn, kết thúc bằng một câu lưu ý rằng đây chỉ là công cụ tham khảo, không thay thế chẩn đoán của chuyên gia. Endpoint /chat dùng lại đúng ngữ cảnh chẩn đoán đó cho các câu hỏi tiếp

theo, kèm tối đa sáu tin nhắn gần nhất trong lịch sử hội thoại để mô hình giữ được mạch trao đổi thay vì trả lời rời rạc từng câu.

Phần này được thiết kế như một tính năng tùy chọn. Khóa API được đọc từ một tệp cấu hình môi trường và không đưa vào mã nguồn; nếu không có khóa, /predict vẫn trả về đầy đủ kết quả chẩn đoán và các bản đồ giải thích, chỉ để trống trường diễn giải, còn giao diện tự ẩn hai khối liên quan. Cách bố trí này giữ cho phần lõi (chẩn đoán và giải thích trực quan) chạy hoàn toàn ngoại tuyến trên máy người dùng, phù hợp với định hướng thiết bị biên của đề tài, còn phần phụ thuộc dịch vụ đám mây có thể tắt đi mà ứng dụng vẫn dùng được. Cần lưu ý một ranh giới khi đánh giá: nội dung do mô hình ngôn ngữ sinh ra không được kiểm chứng bởi bất kỳ nguồn nông học nào trong phạm vi chuyên đề, và độ trễ của nó phụ thuộc vào đường truyền mạng chứ không phải chi phí tính toán của ChiliEdgeNet.

6.1.5. *Đóng gói và triển khai*

Backend và frontend được đóng gói thành hai image Docker riêng, khởi động cùng nhau bằng một tệp docker compose duy nhất. Checkpoint được sao chép thẳng vào image lúc dựng thay vì gắn qua volume, để cùng một image chạy được trên những nền tảng không cho gắn ổ đĩa như Hugging Face Spaces hay Cloud Run. Backend cài bản torch cho CPU; bỏ phần CUDA giúp image nhẹ đi đáng kể. Với cách đóng gói này, người chấm hoặc người dùng thử chỉ cần một lệnh docker compose up để dựng lại toàn bộ hệ thống trên máy của mình mà không phải cài Python, Node hay bất kỳ thư viện nào.

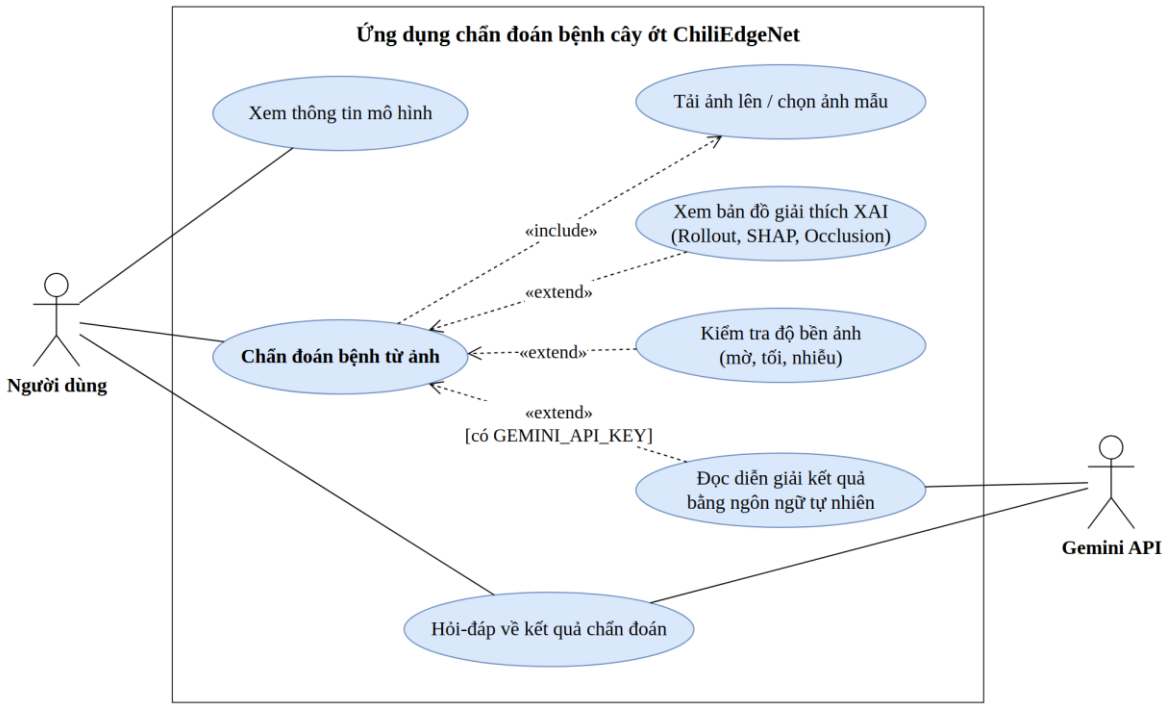
6.2. Phân tích thiết kế hệ thống

Mục 6.1 mô tả ứng dụng theo góc nhìn triển khai (thư mục, dịch vụ, endpoint). Mục này bổ sung góc nhìn phân tích thiết kế bằng ba sơ đồ: biểu đồ use case cho biết người dùng làm được gì với hệ thống, biểu đồ tuần tự cho biết các thành phần phối hợp ra sao trong một lượt chẩn đoán, và biểu đồ luồng dữ liệu (DFD) cho biết dữ liệu nào đi qua đâu, đặc biệt là phần dữ liệu được gửi ra dịch vụ bên ngoài. Biểu đồ lớp và biểu đồ hoạt động không được trình bày riêng: backend chỉ gồm bốn mô-đun

Python tổ chức theo hàm thay vì theo lớp đối tượng, cấu trúc của mô hình đã được mô tả ở Hình 3.2, còn các nhánh điều kiện của luồng xử lý đã được thể hiện bằng các khung opt trong biểu đồ tuần tự. Các sơ đồ được vẽ bằng draw.io, bám theo mã nguồn thực tế của ứng dụng.

6.2.1. *Biểu đồ use case*

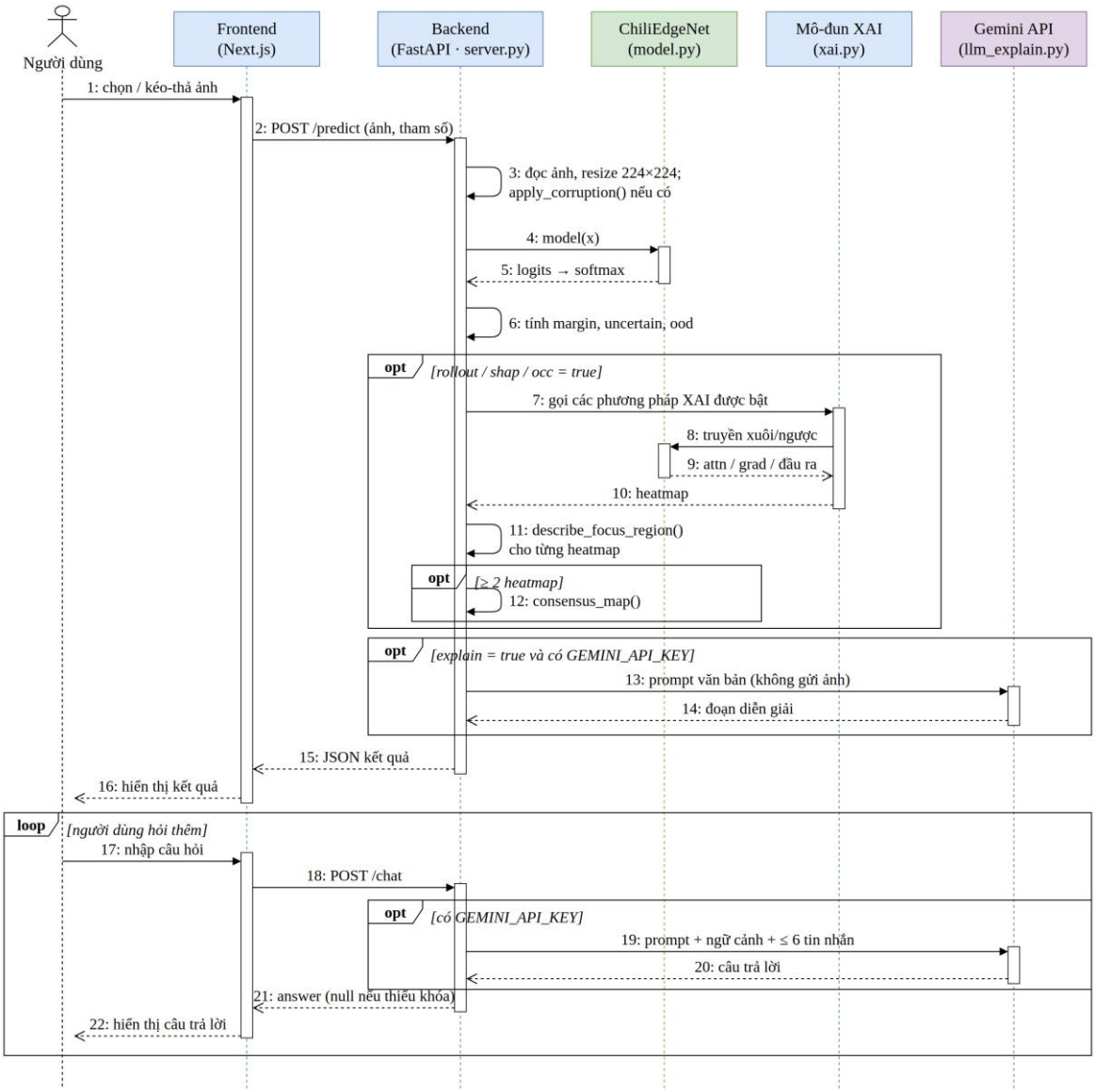
Hình 6.2 trình bày biểu đồ use case. Hệ thống có một tác nhân chính là người dùng (nông dân hoặc cán bộ kỹ thuật) và một tác nhân phụ là Gemini API. Use case trung tâm là chẩn đoán bệnh từ ảnh, luôn bao gồm (include) bước tải ảnh lên hoặc chọn ảnh mẫu. Ba use case xem bản đồ giải thích XAI, kiểm tra độ bền ảnh và đọc diễn giải bằng ngôn ngữ tự nhiên là phần mở rộng (extend) của một lượt chẩn đoán chứ không đứng riêng: bản đồ XAI được tính kèm kết quả, kiểm tra độ bền gửi lại ảnh kèm tham số suy giảm, còn phần diễn giải chỉ khả dụng khi backend được cấu hình khóa GEMINI_API_KEY. Hai use case độc lập còn lại là xem thông tin mô hình (endpoint /info) và hỏi đáp về kết quả chẩn đoán (endpoint /chat).



Hình 6.2. Biểu đồ use case của ứng dụng minh họa.

6.2.2. *Biểu đồ tuần tự*

Hình 6.3 mô tả trình tự thông điệp của một lượt chẩn đoán và các lượt hỏi đáp tiếp theo, bám theo mã nguồn của frontend (page.tsx) và backend (server.py, xai.py, llm_explain.py). Frontend gửi ảnh tới /predict kèm các cờ bật/tắt XAI và phần diễn giải; khi người dùng kéo thanh trượt kiểm tra độ bền, yêu cầu gửi thêm tham số suy giảm và tắt cả XAI lẫn diễn giải. Backend tiền xử lý ảnh, gọi ChiliEdgeNet một lần để lấy vector xác suất, rồi tính margin và hai cờ uncertain, ood (mục 6.1.2). Khung opt thứ nhất là phần XAI: mỗi phương pháp chỉ chạy khi được bật, và mỗi phương pháp gọi lại mô hình theo cách riêng. Attention Rollout chỉ lan truyền xuôi và đọc ma trận chú ý qua hook, Gradient SHAP lan truyền cả xuôi lẫn ngược để lấy gradient, còn Occlusion chỉ lan truyền xuôi nhiều lần trên ảnh bị che từng vùng. Mỗi heatmap được mô tả vùng tập trung bằng describe_focus_region(); bản đồ đồng thuận consensus_map() chỉ được tính khi có từ hai heatmap trở lên. Khung opt thứ hai chỉ chạy khi explain = true và có khóa API; thông điệp gửi tới Gemini chỉ chứa văn bản, không chứa ảnh. Khung loop phía dưới là phần hỏi đáp: mỗi câu hỏi đi qua /chat kèm ngữ cảnh chẩn đoán do trình duyệt lưu, backend chỉ ghép tối đa sáu tin nhắn gần nhất của lịch sử vào prompt; nếu không có khóa API, backend không gọi Gemini mà trả về answer rỗng và giao diện báo LLM không khả dụng.

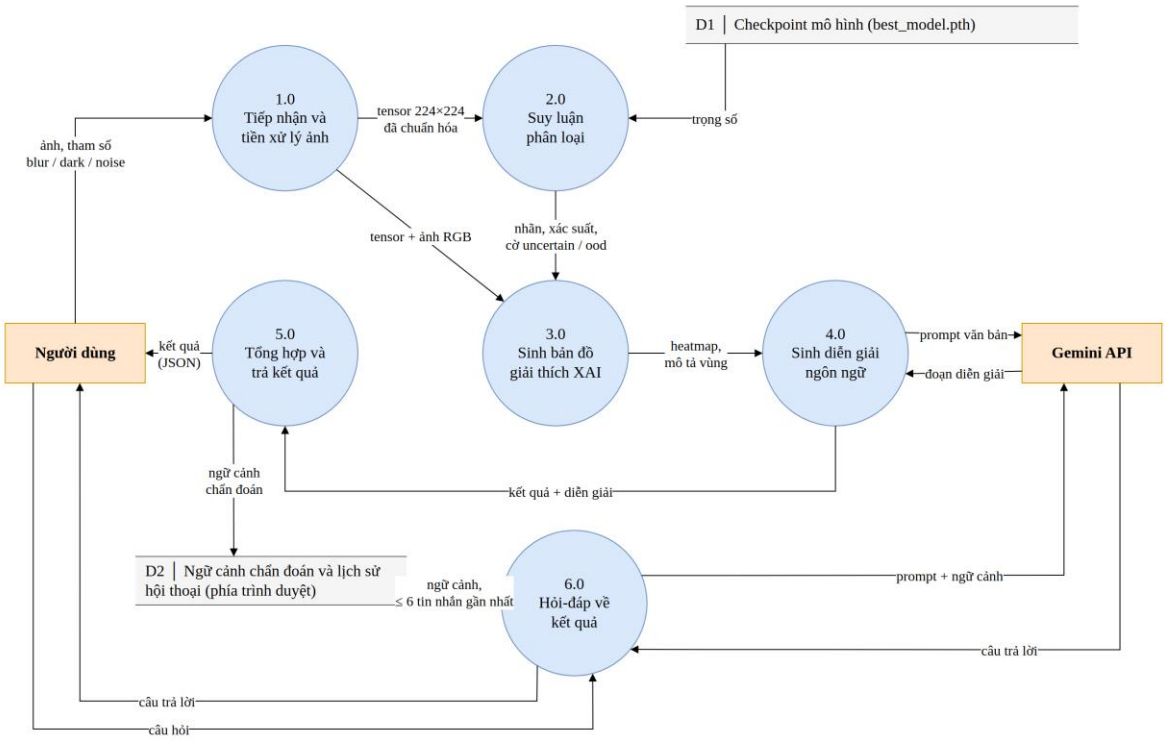


Hình 6.3. Biểu đồ tuần tự của một lượt chẩn đoán (/predict) và hỏi đáp (/chat).

6.2.3. *Biểu đồ luồng dữ liệu*

Hình 6.4 là DFD mức 1 của hệ thống, gồm hai thực thể ngoài (người dùng, Gemini API), sáu tiến trình và hai kho dữ liệu. Kho D1 là tệp checkpoint best_model.pth, được nạp một lần khi backend khởi động. Kho D2 là ngữ cảnh chẩn đoán và lịch sử hội thoại, được giữ ở phía trình duyệt và gửi kèm mỗi yêu cầu /chat, nên backend không cần cơ sở dữ liệu. DFD cho thấy rõ ranh giới dữ liệu với dịch vụ bên ngoài: mọi luồng đi tới Gemini API (từ tiến trình 4.0 và 6.0) chỉ mang văn bản gồm nhãn, xác suất, mô tả vùng XAI và câu hỏi; ảnh gốc chỉ đi qua các tiến trình 1.0

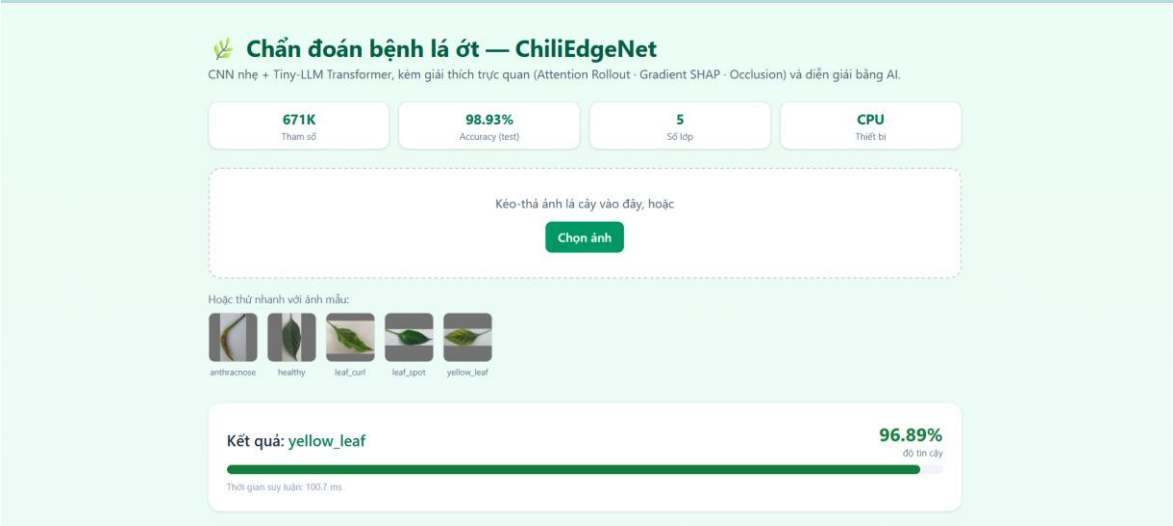
đến 3.0 chạy cục bộ. Các luồng qua tiến trình 3.0 và 4.0 chỉ phát sinh khi XAI và phần diễn giải được bật; khi tắt, kết quả phân loại được chuyển thẳng tới tiến trình 5.0.



Hình 6.4. Biểu đồ luồng dữ liệu (DFD) mức 1 của ứng dụng minh họa.

6.3. Giao diện và luồng sử dụng

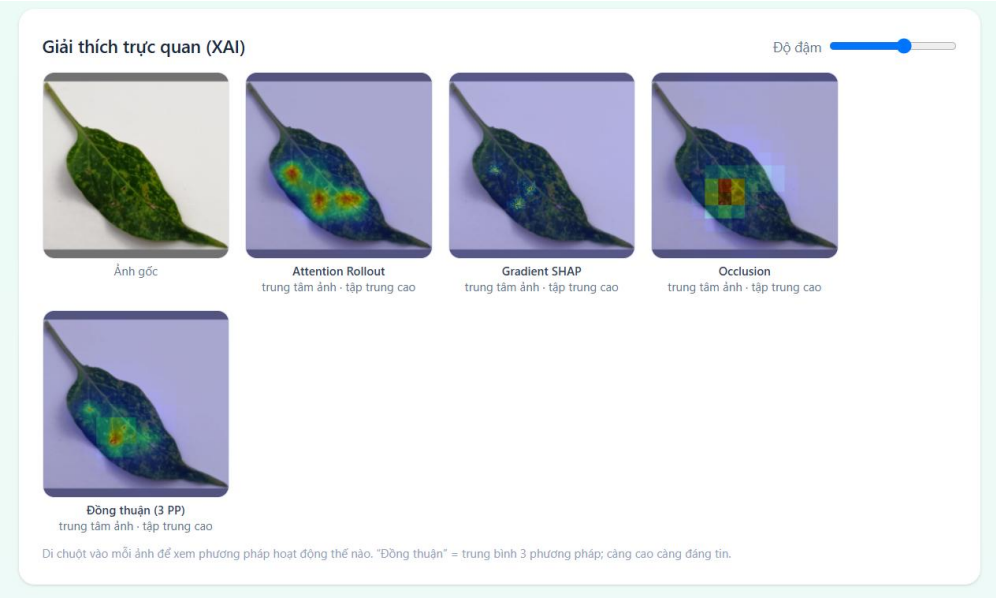
Hình 6.5 đến Hình 6.9 dưới đây là ảnh chụp từ ứng dụng đang chạy, theo đúng luồng thao tác của người dùng cuối.



Hình 6.5. Trang chủ ứng dụng: thẻ thông tin mô hình, khung tải ảnh và kết quả dự đoán.

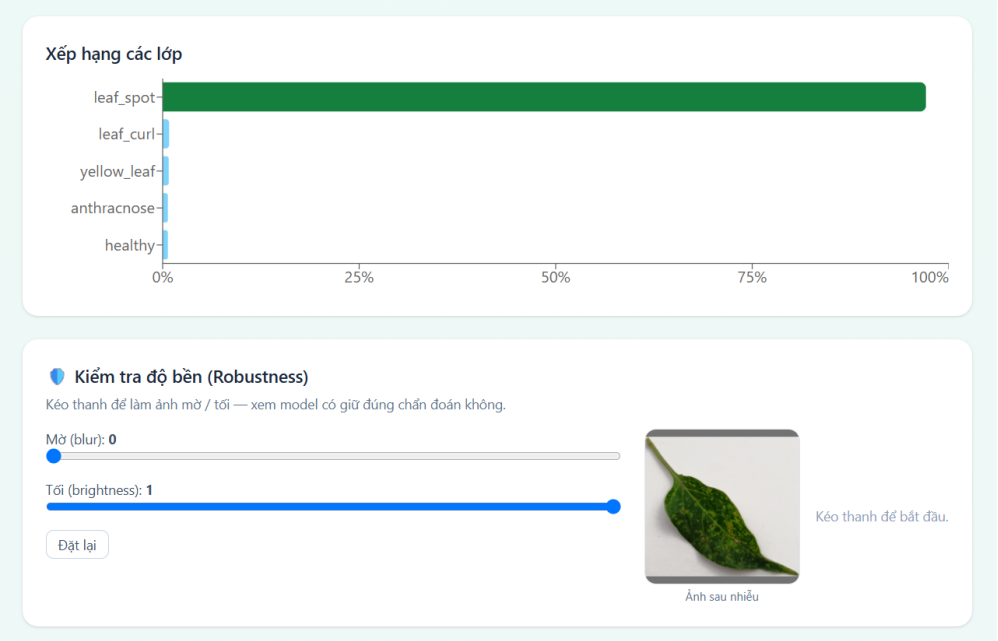
Thẻ thông tin mô hình hiển thị tham số, độ chính xác, số lớp và thiết bị đang chạy; kết quả dự đoán hiển thị kèm độ tin cậy.

Sau khi người dùng tải lên hoặc chọn một ảnh mẫu, hệ thống trả về lớp dự đoán cùng thanh độ tin cậy; nếu độ tin cậy thấp hoặc ảnh có dấu hiệu nằm ngoài phạm vi năm lớp đã huấn luyện, giao diện cảnh báo thay vì đưa ra kết quả sai lệch một cách im lặng.



Hình 6.6. Khối giải thích trực quan (XAI) trên giao diện ứng dụng.

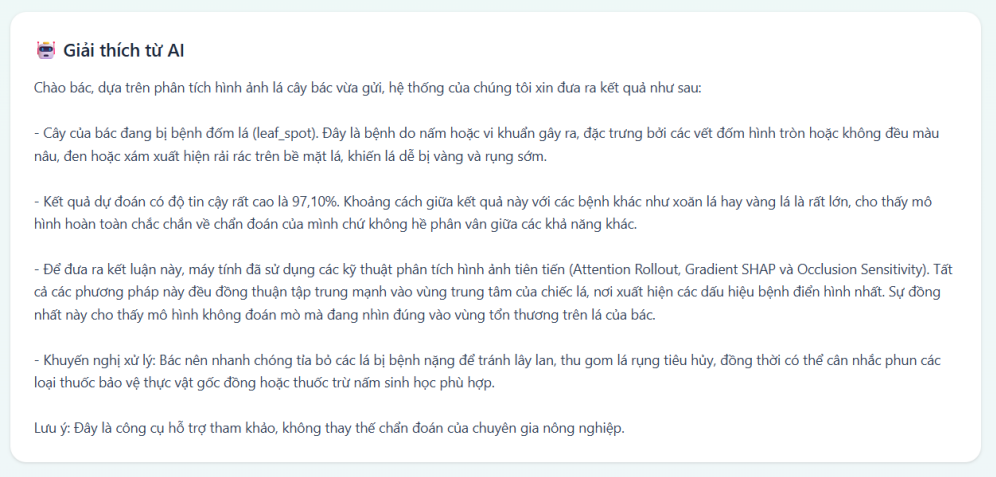
Bốn bản đồ trên tái hiện trực tiếp ba kỹ thuật XAI trình bày ở mục 5.1 (Occlusion Sensitivity, Gradient SHAP, Attention Rollout), cho người dùng cuối, chứ không riêng nhà phát triển, thấy được mô hình đang "nhìn" vào đâu trên chính ảnh họ vừa tải lên.



Hình 6.7. Biểu đồ xếp hạng xác suất theo lớp và khối kiểm tra độ bền (Robustness).

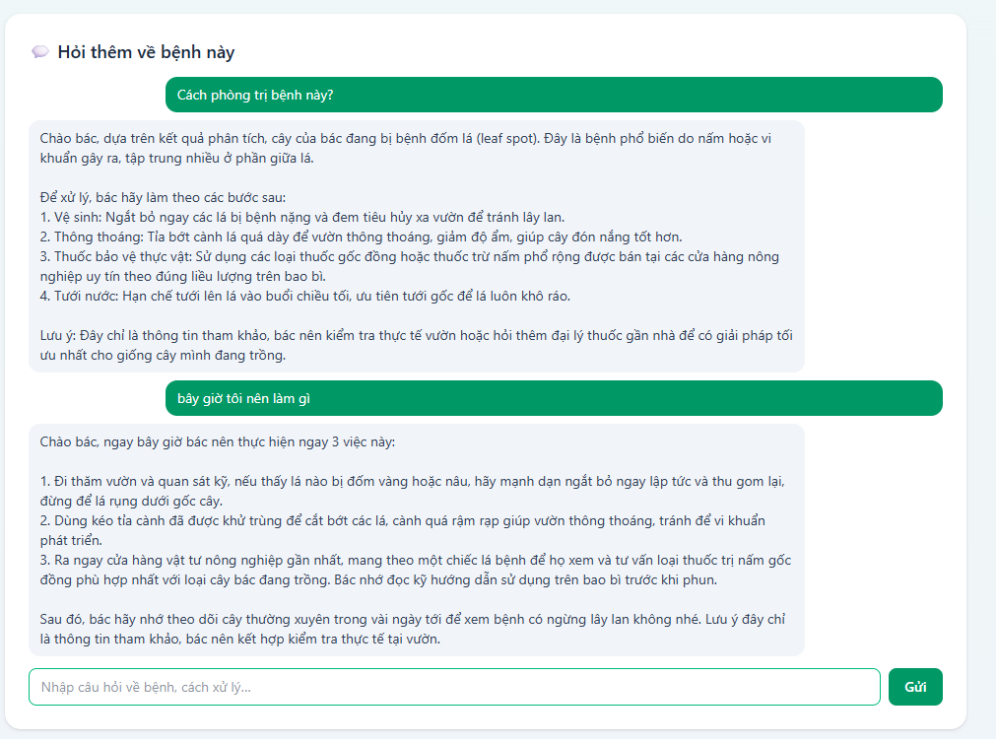
Khối Robustness có hai thanh trượt mờ (blur) và tối (brightness), so sánh trực tiếp dự đoán trên ảnh gốc và ảnh đã làm suy giảm.

Khối Robustness minh họa trực tiếp phát hiện ở mục 4.4: kéo thanh trượt tối làm ảnh giảm sáng nhưng dự đoán của checkpoint augmented vẫn giữ nguyên trong đa số trường hợp nhẹ, tương ứng với mức tăng bền do tăng cường dữ liệu và vai trò của khối Tiny Transformer đã đo định lượng ở Chương 4.



Hình 6.8. Đoạn giải thích kết quả bằng ngôn ngữ tự nhiên do LLM sinh ra.

Đoạn văn được sinh dựa trên lớp dự đoán và vùng tập trung của các bản đồ XAI.



Hình 6.9. Khung hỏi đáp về bệnh cây, hỗ trợ bởi LLM.

Người dùng có thể hỏi thêm về bệnh (cách phòng trị, nguyên nhân...); LLM trả lời dựa trên ngữ cảnh chẩn đoán vừa có được.

Hai tính năng ở Hình 6.8–6.9 minh họa một hướng ứng dụng thực tế mà một mô hình phân loại thuần túy không có: đầu ra là một nhãn, một con số xác suất, cộng thêm lời giải thích và kênh tương tác bằng ngôn ngữ tự nhiên, phù hợp với người dùng cuối là nông dân, những người cần một khuyến nghị xử lý hơn là một con số kỹ thuật. Cần lưu ý rằng bản thân nội dung do LLM sinh ra ở hai khối này chưa được đánh giá định lượng về độ chính xác y khoa/nông học trong chuyên đề, và nên được xem như tính năng hỗ trợ minh họa, không phải một kết quả thực nghiệm.

6.4. Chương trình demo thời gian thực trên camera laptop

Ngoài luồng tải ảnh tĩnh ở mục 6.3, chuyên đề còn viết một chương trình demo riêng chạy trực tiếp trên camera của máy tính xách tay. Mục đích không phải là bổ sung một tính năng cho ứng dụng web, mà là đưa mô hình ra khỏi môi trường notebook và đặt nó trước một nguồn ảnh không được kiểm soát: ánh sáng phòng, nền phía sau, độ rung tay, khoảng cách và góc chụp đều do người dùng quyết định tại chỗ. Vòng lặp webcam và cách vẽ khung quan tâm lên khung hình được tham khảo từ mã nguồn demo mà giảng viên hướng dẫn cung cấp, phần mô hình và tiền xử lý thì dùng lại đúng ChiliEdgeNet của đề tài.

6.4.1. *Cách hoạt động*

Chương trình mở webcam bằng backend DirectShow của OpenCV, vốn khởi động nhanh và ổn định hơn trên Windows, rồi vẽ lên khung hình một khung chữ nhật ở chính giữa với cạnh bằng khoảng một phần ba kích thước khung hình. Chỉ phần ảnh nằm trong khung này, vùng quan tâm (ROI), được cắt ra, chuyển từ BGR sang RGB và đưa vào mô hình; phần nền còn lại bị bỏ qua. Cách làm này vừa cho người dùng biết chính xác cần đặt lá vào đâu, vừa giảm lượng nền không liên quan lọt vào đầu vào của mô hình. Chuỗi tiền xử lý dùng lại đúng ba bước Resize 224×224, ToTensor và Normalize của lúc huấn luyện, nhập trực tiếp từ thư mục model dùng chung với backend nên không có nguy cơ lệch tiền xử lý giữa hai chương trình.

Điểm khác biệt so với các demo webcam thông thường là mô hình không chạy trên mọi khung hình. Video chỉ hiển thị bình thường cho tới khi người dùng bấm

phím cách; lúc đó mô hình chạy đúng một lần trên khung hình tại thời điểm bấm, và kết quả đứng yên trên màn hình cho tới lần bấm kế tiếp. Nếu chạy liên tục, nhãn và độ tin cậy sẽ nhảy theo từng rung tay hay thay đổi ánh sáng, người xem không biết con số nào là con số cần ghi nhận và cũng không thể gán nhãn thật cho một phép đo cụ thể. Chạy theo yêu cầu cho một phép đo rõ ràng ứng với đúng một dòng nhật ký.

6.4.2. *Giao thức đo*

Trước mỗi lần bấm, người dùng gán nhãn thật cho vật đang đặt trong khung bằng các phím số từ 1 đến 5 theo đúng thứ tự năm lớp. Mỗi lần bấm phím cách, chương trình ghi một dòng vào tệp CSV gồm thời điểm, chỉ số khung hình, lớp dự đoán, độ tin cậy, độ trễ tính bằng mili giây, nhãn thật và cờ đúng/sai. Khoảng thời gian được tính bắt đầu từ lúc chuyển vùng quan tâm sang ảnh PIL, qua tiền xử lý, suy luận và softmax; không tính thời gian đọc khung hình từ camera, cắt ROI, chuyển không gian màu, dựng giao diện hay ghi tệp. Mô hình được chạy nóng năm lượt trước khi vào vòng lặp để chi phí khởi tạo của lần gọi đầu tiên không lẫn vào số liệu. Khi thoát, chương trình tự tổng hợp thống kê độ trễ, độ tin cậy trung bình, tỉ lệ dự đoán đúng và ma trận nhầm lẫn trên toàn bộ các lần đã đo.

6.4.3. *Kết quả một phiên đo thực tế*

Bảng 6.1 tổng hợp phiên đo ngày 3 tháng 8 năm 2026 trên CPU Intel Core i7-13650HX, gồm 62 lượt đo.

Bảng 6.1. Tổng hợp phiên demo camera laptop.

Chỉ số	**Giá trị**
Số lượt đo	62
Độ trễ trung bình	32,18 ms
Độ trễ trung vị	29,70 ms
Độ trễ bách phân vị 95	46,49 ms
Độ trễ nhỏ nhất–lớn nhất	17,33–122,76 ms

Về độ trễ, trung vị 29,70 ms và bách phân vị 95 ở mức 46,49 ms (hai con số đã báo cáo ở mục 4.7) cho thấy phần tiền xử lý cộng suy luận đủ nhanh để chạy tương

tác trên CPU laptop mà không cần GPU. Giá trị lớn nhất 122,76 ms rơi đúng vào lần đo đầu tiên của phiên, phần còn lại hầu hết nằm dưới 50 ms.

Chương 7. HẠN CHẾ VÀ HƯỚNG PHÁT TRIỂN

Chương này tổng hợp các giới hạn của chuyên đề, từ dữ liệu, phương pháp đánh giá đến phần cứng dùng để đo độ trễ (mục 7.1), và các hướng phát triển tương ứng (mục 7.2).

7.1. Hạn chế

Năng lực phân giải thống kê của tập test. Với 373 ảnh, một ảnh trị giá 0,27 điểm phần trăm, và mô hình đề xuất sai trên 4 ảnh. Chênh lệch 0,80 điểm phần trăm (3 ảnh) so với kiến trúc nền tảng [7] nằm trong dải nhiễu của một lần chia dữ liệu duy nhất: khoảng tin cậy Wilson 95% của ChiliEdgeNet [97,28; 99,58] chồng lấn với khoảng [96,18; 99,09] của kiến trúc nền tảng. Cả hai mô hình đều khớp seed 42 nhưng mỗi mô hình chỉ chạy một lần. Hơn nữa, RTR_Lite_MobileNet [7] được chọn làm baseline chính sau khi xem xét kết quả test của cả nhóm baseline, nên phép so sánh này mang tính khám phá, không cấu thành một đánh giá test độc lập. Để thiết lập một xếp hạng dứt khoát cần lựa chọn qua validation rồi một lần đánh giá test duy nhất, nhiều seed, và/hoặc một kiểm định McNemar trực tiếp giữa hai tập dự đoán.

Số lần chạy và năng lực thống kê. Mô hình đề xuất và toàn bộ ablation đều đến từ một lần chạy seed 42. Chỉ hai kết quả còn ý nghĩa sau hiệu chỉnh Holm: tác động của kết nối tắt trên ảnh sạch và ưu thế của khối Tiny Transformer ở mức giảm sáng 0,2 (mục 4.3, 4.4). Các chênh lệch còn lại, kể cả ba phép so sánh Triplet/RES_Block, mới dừng ở mức xu hướng và cần được xác nhận qua nhiều lần chạy.

Phạm vi của stress test. Thí nghiệm chỉ khảo sát bốn mức nhòe Gaussian và bốn hệ số giảm sáng tổng hợp từ chính tập test. Thiết kế này chưa bao phủ nhiễu mức cảm biến (sensor noise), nén JPEG, nhòe chuyển động (motion blur), hay ảnh thực chụp dưới điều kiện thiếu sáng thật; kết quả không nên được suy rộng cho mọi điều kiện chụp.

Khoảng cách miền và phạm vi của đánh giá few-shot. Đánh giá chỉ dùng bốn nhãn của bộ dữ liệu Mendeley Data có tương ứng trực tiếp với healthy, leaf_curl, leaf_spot và yellow_leaf; không đánh giá anthracnose và loại trừ hai nhãn nguồn mà mô hình không hỗ trợ. Trên tập truy vấn sạch 1.463 ảnh, với giao bằng không giữa tập hỗ trợ và tập truy vấn (xác nhận bằng băm), độ chính xác đạt 30,96% ở checkpoint gốc và 89,75% sau thích nghi few-shot. Cả hai checkpoint đều huấn luyện với seed 42, cùng kiến trúc, cùng siêu tham số cơ sở; hồ sơ huấn luyện vẫn cần được củng cố thêm bằng log thực thi hoặc file cấu hình đầy đủ. Vì chỉ dùng một bộ dữ liệu ngoài miền và một tập hỗ trợ cố định, cải thiện này chưa chứng minh giao thức few-shot tổng quát hóa được sang các nguồn dữ liệu khác hay các phương pháp chọn tập hỗ trợ khác.

Phần cứng. Con số 20,24 ms/ảnh được đo trên CPU máy ảo Kaggle hạng máy chủ, không phải phần cứng biên. INT8 và FP16 trong lần chạy hiện tại giảm được dung lượng lưu trữ nhưng chưa cho thấy tăng tốc độ suy luận (xem thêm khảo sát mở rộng ở mục 4.8); việc đối chuẩn trên điện thoại hoặc máy tính nhúng với backend triển khai thật vẫn còn cần thiết.

7.2. Hướng phát triển

Từ các hạn chế trên, chuyên đề này đề xuất năm hướng phát triển tiếp theo:

1. Lặp lại thực nghiệm trên nhiều seed để các kết luận ablation ở Chương 4 có thể được xác nhận chắc chắn hơn thay vì dừng ở mức quan sát định hướng thiết kế từ một lần chạy.
2. Kiểm tra ảnh trùng lặp/gần trùng lặp giữa các tập train/val/test, nhằm loại trừ khả năng rò rỉ dữ liệu âm thầm làm lệch các giá trị độ chính xác đã báo cáo.
3. Mở rộng stress test sang các suy giảm thu nhận ảnh thật: nhiễu mức cảm biến, nén JPEG, nhòe chuyển động, và ảnh chụp thật dưới điều kiện thiếu sáng, thay vì chỉ dựa trên suy giảm tổng hợp.

4. Đo độ trễ và bộ nhớ trên phần cứng biên vật lý (điện thoại thông minh, board nhúng như Raspberry Pi hoặc Jetson Nano) với một backend lượng tử hóa triển khai thật, thay vì chỉ đo trên CPU/GPU máy chủ ảo.

5. Mở rộng đánh giá chuyển miền sang nhiều bộ dữ liệu độc lập hơn và nhiều phương pháp chọn tập hỗ trợ few-shot khác nhau, đồng thời bổ sung ảnh nền tự nhiên (ngoài đồng ruộng) để định lượng đầy đủ hơn khoảng cách miền với điều kiện thu thập có kiểm soát của ChiliEdge-DS.

KẾT LUẬN

Chuyên đề đề xuất ChiliEdgeNet, một mô hình lai CNN-Transformer 671.173 tham số, đạt 98,93% độ chính xác trên bài toán nhận diện bệnh cây ớt và chạy ngoại tuyến trên CPU. So với kiến trúc nền tảng RTR_Lite_MobileNet [7], hai lợi ích cần được đọc khác nhau. Khoản giảm khoảng 35% tham số là một phép đếm chính xác. Mức tăng 0,80 điểm phần trăm độ chính xác thì chỉ là xu hướng, vì mỗi mô hình chạy một lần và hai khoảng tin cậy chồng lấn nhau.

Về phương pháp, chuyên đề kiểm tra từng thành phần thiết kế bằng ablation huấn luyện lại từ đầu, kiểm định ghép cặp có hiệu chỉnh Holm và một đánh giá chuyển miền trên bộ dữ liệu độc lập. Có bốn kết quả chính:

1. **Trên ảnh sạch, kết nối tắt của MobileNetV2 có tác động lớn hơn khối Tiny Transformer.** Vô hiệu hóa các kết nối tắt trong xương sống làm giảm 2,41 điểm phần trăm độ chính xác trên ảnh sạch, có ý nghĩa thống kê sau hiệu chỉnh Holm ($p_{Holm} = 0,0234$). Ngược lại, trên ảnh sạch, việc bỏ khối Tiny Transformer không cho thấy thiệt hại đo được; ưu thế của khối này chỉ xuất hiện và đạt ý nghĩa thống kê ở mức suy giảm ánh sáng nghiêm trọng nhất (giảm sáng ×0,2: +3,75 điểm phần trăm, $p_{Holm} = 0,0105$). Khối này có vẻ giúp giữ biểu diễn khi ảnh suy giảm mạnh hơn là tăng độ chính xác trên ảnh sạch.
2. **Ở lần chạy này, ba kỹ thuật tiết kiệm tham số từ LLM không làm giảm độ chính xác trên ảnh sạch.** 2D-RoPE, GQA và RMSNorm lần lượt tiết kiệm 25.088, 16.384 và 384 tham số so với phương án tiêu chuẩn (PE học được, MHA, LayerNorm); trên ảnh sạch, cả ba biến thể tiêu chuẩn không vượt trội đáng kể so với ChiliEdgeNet, và McNemar không cho thấy ý nghĩa thống kê ở bất kỳ so sánh nào (mục 4.5).
3. **Hai khối kế thừa Triplet Attention và RES_Block không mang lại lợi ích đo được.** Với thiết kế thêm lại kiểu 2×2, không phép so sánh nào liên quan đến Triplet Attention và RES_Block đạt ý nghĩa sau hiệu chỉnh Holm; kết quả ủng hộ việc loại bỏ cả hai khối để giảm độ phức tạp không cần thiết mà không

mất độ chính xác đo được, dù không chứng minh các kiến trúc là tương đương.

4. **Một tập hỗ trợ nhỏ đủ để cải thiện mạnh kết quả chuyển miền.** Trên tập truy vấn liên bộ dữ liệu 1.463 ảnh từ Bangladesh, độ chính xác tăng từ 30,96% lên 89,75% (+58,78 điểm phần trăm) sau khi thích nghi bằng 42 ảnh hỗ trợ, với 870 ảnh chuyển từ sai sang đúng và chỉ 10 ảnh theo chiều ngược lại ($p = 1{,}83 \times 10^{-242}$). Mức cải thiện này lớn hơn một bậc so với tác động của bất kỳ thành phần kiến trúc nào ở Chương 4, dù chỉ áp dụng cho bốn lớp được ánh xạ và một bộ dữ liệu ngoài miền.

Điểm cắt xương sống tại features[0:14] giữ được lưới đặc trưng 14×14 (196 token), nhờ đó bản đồ Attention Rollout ở Chương 5 đủ mịn để khoanh vùng triệu chứng. Trên năm ảnh minh họa, vùng phản hồi cao của ba kỹ thuật XAI nhìn chung trùng với lá, quả hoặc vùng bệnh. Trên toàn bộ 373 ảnh test, ChiliEdgeNet đạt 98,93% so với 78,55% của Gemini 3.1 Flash Lite chạy zero-shot. Độ trễ ở mức hệ thống thấp hơn khoảng 105 lần, dù đây là so sánh giữa hai cách triển khai (mục 5.2). Mục 4.7 và Chương 6 cho thấy mô hình tích hợp được vào một ứng dụng camera, với độ trễ trung vị dưới 30 ms trên CPU laptop. Các hướng phát triển ở mục 7.2 gồm xác nhận kết quả trên nhiều seed, thử với suy giảm thu nhận ảnh thật, đo trên phần cứng biên vật lý và mở rộng đánh giá chuyển miền.

TÀI LIỆU THAM KHẢO

[1] S. Abnar and W. Zuidema, "Quantifying attention flow in transformers," in *Proc. Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2020, pp. 4190-4197. DOI: 10.18653/v1/2020.acl-main.385.

[2] J. Ahmmed, M. A. Kabir, A. ur Rehman, and A. Bermak, "A lightweight hybrid CNN and transformer model for medicinal leaf disease classification with explainable AI," *Scientific Reports*, vol. 16, art. no. 8243, 2026. DOI: 10.1038/s41598-026-39182-3.

[3] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebron, and S. Sanghai, "GQA: Training generalized multi-query transformer models from multi-head checkpoints," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2023, pp. 4895-4901. DOI: 10.18653/v1/2023.emnlp-main.298.

[4] M. Ali, M. Salma, M. El Haji, and B. Jamal, "Plant disease detection using vision transformers," *International Journal of Electrical and Computer Engineering (IJECE)*, vol. 15, no. 2, pp. 2334-2344, 2025. DOI: 10.11591/ijece.v15i2.pp2334-2344.

[5] S. Bhagat, M. Kokare, V. Haswani, P. Hambarde, T. Taori, P. H. Ghante, and D. K. Patil, "Advancing real-time plant disease detection: A lightweight deep learning approach and novel dataset for pigeon pea crop," *Smart Agricultural Technology*, vol. 7, art. no. 100408, 2024. DOI: 10.1016/j.atech.2024.100408.

[6] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, et al., "An image is worth 16x16 words: Transformers for image recognition at scale," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2021. [Online]. Available: https://arxiv.org/abs/2010.11929.

[7] S. Duhan, P. Gulia, N. S. Gill, and E. Narwal, "RTR_Lite_MobileNetV2: A lightweight and efficient model for plant disease detection and classification,"

Current Plant Biology, vol. 42, art. no. 100459, 2025. DOI: 10.1016/j.cpb.2025.100459.

[8] G. Erion, J. D. Janizek, P. Sturmfels, S. M. Lundberg, and S.-I. Lee, "Improving performance of deep learning models with axiomatic attribution priors and expected gradients," *Nature Machine Intelligence*, vol. 3, no. 7, pp. 620-631, 2021. DOI: 10.1038/s42256-021-00343-w.

[9] P. Gole, P. Bedi, S. Marwaha, M. A. Haque, and C. K. Deb, "TrIncNet: A lightweight vision transformer network for identification of plant diseases," *Frontiers in Plant Science*, vol. 14, art. no. 1221557, 2023. DOI: 10.3389/fpls.2023.1221557.

[10] Google DeepMind, "Gemini 3.1 Flash-Lite: Model Card," 2026. [Online]. Available: https://deepmind.google/models/model-cards/gemini-3-1-flash-lite/ (accessed Jul. 8, 2026).

[11] T. M. Hải, "Chili Leaf Disease Cross-Dataset Query Set," Kaggle dataset, version 2, 2026. [Online]. Available: https://www.kaggle.com/datasets/tminhhi/chili-leaf-disease-cross-dataset-query-set/version/2.

[12] T. M. Hải, "Chili Leaf Disease Few-Shot," Kaggle dataset, version 1, 2026. [Online]. Available: https://www.kaggle.com/datasets/tminhhi/chili-leaf-disease-few-shot/version/1.

[13] T. M. Hải, "ChiliEdge-DS: Chili Pepper Leaf Disease Dataset," version 1.0, 2026. [Online]. Available: Google Drive, https://drive.google.com/drive/folders/1DesS0c5xT6J72tIv8AMZLwC8ANFUk2PC?usp=sharing.

[14] S. M. Hassan, K. S. Roy, R. A. Hazarika, M. Alam, and M. Mukherjee, "Multi-kernel inception-enhanced vision transformer for plant leaf disease recognition," *Scientific Reports*, vol. 15, art. no. 30997, 2025. DOI: 10.1038/s41598-025-16142-x.

[15] S. Hemalatha and J. J. B. Jayachandran, "A multitask learning-based vision transformer for plant disease localization and classification," *International Journal of Computational Intelligence Systems*, vol. 17, no. 1, art. no. 188, 2024. DOI: 10.1007/s44196-024-00597-3.

[16] B. Heo, S. Park, D. Han, and S. Yun, "Rotary position embedding for Vision Transformer," in *Computer Vision – ECCV 2024*, Lecture Notes in Computer Science, vol. 15068, 2025, pp. 289-305. DOI: 10.1007/978-3-031-72684-2_17.

[17] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65-70, 1979. [Online]. Available: https://www.jstor.org/stable/4615733.

[18] J. Hu, L. Shen, and G. Sun, "Squeeze-and-excitation networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018, pp. 7132-7141. DOI: 10.1109/CVPR.2018.00745.

[19] S. Jia, G. Wang, H. Li, Y. Liu, L. Shi, and S. Yang, "ConvTransNet-S: A CNN-Transformer hybrid disease recognition model for complex field environments," *Plants*, vol. 14, no. 15, art. no. 2252, 2025. DOI: 10.3390/plants14152252.

[20] D. S. Joseph, P. M. Pawar, and K. Chakradeo, "Real-time plant disease dataset development and detection of plant disease using deep learning," *IEEE Access*, vol. 12, pp. 16310-16333, 2024. DOI: 10.1109/ACCESS.2024.3358333.

[21] K. R. Kanaparthi and S. S. Ilango, "Deep learning framework for timely detection and classification of chili leaf diseases and pests," *Scientific Reports*, vol. 16, art. no. 4360, 2026. DOI: 10.1038/s41598-025-34477-3.

[22] R. Karthik, R. Menaka, S. Ompirakash, P. Bala Murugan, M. Meenakashi, S. Lingaswamy, and D. Won, "GrapeLeafNet: A dual-track feature fusion network with Inception-ResNet and Shuffle-Transformer for accurate grape leaf disease identification," *IEEE Access*, vol. 12, pp. 19612-19624, 2024. DOI: 10.1109/ACCESS.2024.3361044.

[23] G. Li, Y. Wang, Q. Zhao, P. Yuan, and B. Chang, "PMVT: A lightweight vision transformer for plant disease identification on mobile devices," *Frontiers in Plant Science*, vol. 14, art. no. 1256773, 2023. DOI: 10.3389/fpls.2023.1256773.

[24] R. Maurya, S. Mahapatra, and L. Rajput, "A lightweight meta-ensemble approach for plant disease detection suitable for IoT-based environments," *IEEE Access*, vol. 12, pp. 28096-28108, 2024. DOI: 10.1109/ACCESS.2024.3367443.

[25] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153-157, 1947. DOI: 10.1007/BF02295996.

[26] S. Mehdipour, S. A. Mirroshandel, and S. A. Tabatabaei, "A novel lightweight hybrid CNN-ViT for maize leaf disease classification," *Scientific Reports*, vol. 16, art. no. 10468, 2026. DOI: 10.1038/s41598-026-41190-2.

[27] U. Mishra, A. Pandey, G. Logeswari, and K. Tamilarasi, "Deep learning-based disease detection in potato and mango leaves: A comparative study of CNN, AlexNet, ResNet, and EfficientNet," *Scientific Reports*, vol. 16, art. no. 2788, 2026. DOI: 10.1038/s41598-025-32607-5.

[28] D. Misra, T. Nalamada, A. U. Arasanipalai, and Q. Hou, "Rotate to attend: Convolutional triplet attention module," in *Proc. IEEE Winter Conf. Appl. Comput. Vis. (WACV)*, 2021, pp. 3138-3147. DOI: 10.1109/WACV48630.2021.00318.

[29] M. A. S. Nirob, A. K. M. F. K. Siam, P. Bishshash, and M. Assaduzzaman, "Chili Plant Leaf Disease and Growth Stage Dataset from Bangladesh, version 1," Mendeley Data, 2025. DOI: 10.17632/w9mr3vf56s.1. License: CC BY 4.0.

[30] M. G. F. Odounfa, C. G. Hounmenou, V. K. Salako, A. Affokpon, and R. L. Glèlè Kakaï, "Deep learning enables precision agriculture for sustainable chili

pepper disease detection in Benin," *Discover Artificial Intelligence*, vol. 5, art. no. 315, 2025. DOI: 10.1007/s44163-025-00583-4.

[31] J. Rahaman, P. Paul, A. Chowdhury, and M. Quamruzzaman, "A customized MobileNetV3Large-based deep learning framework for plant disease detection," *Discover Artificial Intelligence*, vol. 6, art. no. 110, 2026. DOI: 10.1007/s44163-025-00733-8.

[32] M. M. Rahman, M. N. Gony, M. S. Ullah, S. M. K. Shuvra, R. Haque, M. R. Ahmed, et al., "MaizeFormerX: A lightweight vision transformer with cross-scale attention for explainable maize leaf disease diagnosis," *Scientific Reports*, vol. 16, art. no. 15160, 2026. DOI: 10.1038/s41598-026-44550-0.

[33] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018, pp. 4510-4520. DOI: 10.1109/CVPR.2018.00474.

[34] J. Su, M. Ahmed, Y. Lu, S. Pan, B. Wen, and Y. Liu, "RoFormer: Enhanced transformer with rotary position embedding," *Neurocomputing*, vol. 568, art. no. 127063, 2024. DOI: 10.1016/j.neucom.2023.127063.

[35] R. Sujatha, S. Krishnan, J. M. Chatterjee, and A. H. Gandomi, "Advancing plant leaf disease detection integrating machine learning and deep learning," *Scientific Reports*, vol. 15, art. no. 11552, 2025. DOI: 10.1038/s41598-024-72197-2.

[36] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS 30)*, 2017, pp. 5998-6008. [Online]. Available: https://arxiv.org/abs/1706.03762.

[37] B. Wang, C. Zhang, Y. Li, C. Cao, D. Huang, and Y. Gong, "An ultra-lightweight efficient network for image-based plant disease and pest infection

detection," *Precision Agriculture*, vol. 24, no. 5, pp. 1836-1861, 2023. DOI: 10.1007/s11119-023-10020-0.

[38] Q. Wang, B. Wu, P. Zhu, P. Li, W. Zuo, and Q. Hu, "ECA-Net: Efficient channel attention for deep convolutional neural networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2020, pp. 11531-11539. DOI: 10.1109/CVPR42600.2020.01155.

[39] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209-212, 1927. DOI: 10.1080/01621459.1927.10502953.

[40] B. Yang, M. Li, F. Li, Y. Wang, Q. Liang, R. Zhao, et al., "A novel plant type, leaf disease and severity identification framework using CNN and transformer with multi-label method," *Scientific Reports*, vol. 14, art. no. 11664, 2024. DOI: 10.1038/s41598-024-62452-x.

[41] S. Yang, Q. Feng, J. Zhang, W. Yang, W. Zhou, and W. Yan, "From laboratory to field: cross-domain few-shot learning for crop disease identification in the field," *Frontiers in Plant Science*, vol. 15, art. no. 1434222, 2024. DOI: 10.3389/fpls.2024.1434222.

[42] M. D. Zeiler and R. Fergus, "Visualizing and understanding convolutional networks," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2014, pp. 818-833. DOI: 10.1007/978-3-319-10590-1_53.

[43] B. Zhang and R. Sennrich, "Root mean square layer normalization," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS 32)*, 2019, pp. 12360-12371. [Online]. Available: https://arxiv.org/abs/1910.07467.

[44] C. Zhao, T. Xu, Z. Zhang, and X. Geng, "Lightweight cross-domain few-shot plant disease recognition through target-domain statistical calibration," *Sensors*, vol. 26, no. 12, art. no. 3632, 2026. DOI: 10.3390/s26123632.